

Let us first consider the computation of l_{ij} . In practice the computed values $\hat{l}_{ik}$ and $\hat{u}_{kj}$ are used, and further roundoff errors occur. Thus the computed $\hat{l}_{ij}$ satisfies

$$\hat{l}_{ij} = \text{fl} \left(\frac{a_{ij} - \sum_{k=1}^{j-1} \hat{l}_{ik} \hat{u}_{kj}}{\hat{u}_{jj}} \right).$$

Different versions of Gaussian elimination accumulate the sum in the numerator in different ways. Proposition 2.7.1 shows that no matter how it is done,

$$\hat{l}_{ij} = \frac{a_{ij}(1 + \gamma_{ij}) - \sum_{k=1}^{j-1} \hat{l}_{ik} \hat{u}_{kj} (1 + \alpha_k)(1 + \gamma_{ik})}{\hat{u}_{jj}} (1 + \beta), \quad (2.7.20)$$

where $|\gamma_{ij}| \leq (j-1)u + O(u^2)$. The quantities α_k and β are the roundoff errors associated with the multiplications and the division, respectively, and they satisfy $|\alpha_k| \leq u$ and $|\beta| \leq u$.

Proceeding just as in the proof of Theorem 2.7.2, we divide the numerator and denominator in (2.7.20) by $(1 + \gamma_{ij})(1 + \beta)$ to remove the error from the a_{ij} term. This move is not strictly necessary, but it yields a more elegant result. We then simplify the resulting equation by consolidating the errors. Define δ_{ik} by

$$1 + \delta_{ik} = \begin{cases} \frac{(1 + \alpha_k)(1 + \gamma_{ik})}{1 + \gamma_{ij}} & \text{if } k < j, \\ \frac{1}{(1 + \gamma_{ij})(1 + \beta)} & \text{if } k = j. \end{cases}$$

Recalling that $1/(1 + \gamma_{ij}) = 1 - \gamma_{ij} + \gamma_{ij}^2 - \gamma_{ij}^3 + \dots$, we have, for $k < j$, $\delta_{ik} = \alpha_k + \gamma_{ik} - \gamma_{ij} + O(u^2)$, and

$$\begin{aligned} |\delta_{ik}| &\leq |\alpha_k| + |\gamma_{ik}| + |\gamma_{ij}| + O(u^2) \\ &\leq u + (j-1)u + O(u^2) + (j-1)u + O(u^2) = (2j-1)u + O(u^2). \end{aligned}$$

Similarly, $\delta_{ij} = -\gamma_{ij} - \beta + O(u^2)$, and

$$\begin{aligned} |\delta_{ij}| &\leq |\gamma_{ij}| + |\beta| + O(u^2) \\ &\leq (j-1)u + O(u^2) + u + O(u^2) = ju + O(u^2). \end{aligned}$$

Thus

$$|\delta_{ik}| \leq 2nu + O(u^2), \quad k = 1, \dots, j.$$

In terms of δ_{ik} , (2.7.20) becomes

$$\hat{l}_{ij} = \frac{a_{ij} - \sum_{k=1}^{j-1} \hat{l}_{ik} \hat{u}_{kj} (1 + \delta_{ik})}{\hat{u}_{jj} (1 + \delta_{ij})}.$$

Multiplying through by $\hat{u}_{jj}(1 + \delta_{ij})$ and rewriting the resulting expression so that all of the error terms are consolidated into a single term e_{ij} , we have

$$a_{ij} + e_{ij} = \sum_{k=1}^j \hat{l}_{ik} \hat{u}_{kj} \quad (2.7.21)$$

for $i > j$ (because we started from (2.7.18), which holds only for $i > j$), where

$$e_{ij} = - \sum_{k=1}^j \hat{l}_{ik} \hat{u}_{kj} \delta_{ik}.$$

Since $|\delta_{ik}| \leq 2nu + O(u^2)$,

$$|e_{ij}| \leq 2nu \sum_{k=1}^j |\hat{l}_{ik}| |\hat{u}_{kj}| + O(u^2) \quad (2.7.22)$$

for $i > j$.

We obtained (2.7.21) and (2.7.22) for $i > j$ by starting from (2.7.18). The same results can be obtained for $i \leq j$ by performing a similar analysis starting from (2.7.19). Thus (2.7.21) and (2.7.22) hold for all i and j . Writing (2.7.21) as a matrix equation, we have

$$A + E = \hat{L}\hat{U}.$$

Writing (2.7.22) as a matrix inequality, we have

$$|E| \leq 2nu |\hat{L}| |\hat{U}| + O(u^2).$$

The bound on $\|E\|_\infty$ follows immediately from the bound on $|E|$, using the properties of the matrix ∞ -norm established in Exercise 2.7.11. $\square$

Exercise 2.7.23 Starting from (2.7.19), demonstrate that (2.7.21) and (2.7.22) hold for $i \leq j$. $\square$

Proof of Theorem 2.7.14. In the forward substitution phase we compute $\hat{y}$ such that $(\hat{L} + \delta\hat{L})\hat{y} = b$, where

$$|\delta\hat{L}| \leq 2nu |\hat{L}|$$

by Theorem 2.7.2. In the back substitution phase we compute $\hat{x}$ such that $(\hat{U} + \delta\hat{U})\hat{x} = \hat{y}$, where

$$|\delta\hat{U}| \leq 2nu |\hat{U}|$$

by the upper-triangular analogue of Theorem 2.7.2. Thus $\hat{x}$ satisfies exactly

$$(\hat{L} + \delta\hat{L})(\hat{U} + \delta\hat{U})\hat{x} = b.$$

Multiplying out the product of matrices and using the fact (Theorem 2.7.13) that $\hat{L}\hat{U} = A + E$, where $|E| \leq 2nu |\hat{L}| |\hat{U}| + O(u^2)$, we have

$$(A + \delta A)\hat{x} = b,$$

where

$$\delta A = E + (\delta\hat{L})\hat{U} + \hat{L}(\delta\hat{U}) + (\delta\hat{L})(\delta\hat{U}).$$

Applying the upper bounds that we have for E , $\delta\hat{L}$, and $\delta\hat{U}$, we obtain

$$|\delta A| \leq 6nu |\hat{L}| |\hat{U}| + O(u^2).$$

The bound on $\|\delta A\|_\infty$ follows immediately from this bound. $\square$

Additional Exercises

Exercise 2.7.24 In Proposition 2.7.1 we showed that summation is backward stable, regardless of the order. In the notation of Proposition 2.7.1 the backward errors γ_j are bounded by $|\gamma_j| \leq (n-1)u + O(u^2)$. In this exercise we obtain tighter bounds on $|\gamma_j|$ for two specific orders of summation of $\sum_{j=1}^n w_j$.

- (a) First we consider the “obvious” order, reversed for notational convenience. Show that if we perform the computation in the order

$$(\cdots(((w_n + w_{n-1}) + w_{n-2})) + \cdots + w_2) + w_1$$

in floating-point arithmetic, then

$$|\gamma_j| \leq \begin{cases} ju + O(u^2) & \text{if } j < n, \\ (n-1)u + O(u^2) & \text{if } j = n. \end{cases}$$

(Look at small cases like $n = 3$ and $n = 4$, and observe the pattern.)

- (b) Now consider summing by pairs. Calculate $w_1 + w_2$, $w_3 + w_4$, $w_5 + w_6$, and so on. If there is an odd term, just let it sit. Now you have a new list, which you can sum by pairs. Keep summing by pairs until you have a single sum. This is easiest to discuss when n is a power of 2, say $n = 2^k$. In this case, how many additions does each term participate in? Express your answer in terms of n . Show that if we carry out this process in floating-point arithmetic, we have

$$|\gamma_j| \leq (\log_2 n)u + O(u^2).$$

This is an excellent result, as $\log_2 n$ grows much more slowly than n . $\square$

Exercise 2.7.25 In this exercise you will assess the backward stability of Gaussian elimination by calculating backward error in the LU decomposition: $E = \hat{L}\hat{U} - A$. Write a MATLAB program that does Gaussian elimination without pivoting, for example

```
A = randn(n);
L = zeros(n); U = zeros(n);
for k = 1:n
    U(k,k:n) = a(k,k:n) - L(k,1:k-1)*U(1:k-1,k:n);
```

```

L(k+1:n,k) = (a(k+1:n,k) - ...
               L(k+1:n,1:k-1)*U(1:k-1,k))/U(k,k);
end
L = L + eye(n);
    
```

- (a) Calculate the LU decomposition of random matrices ($A = \text{randn}(n);$) for several choices of n (e.g. $n = 40, 80, 160$), and note $\|L\|_\infty$, $\|U\|_\infty$, and the norm of the backward error: $\|E\|_\infty = \|LU - A\|_\infty$. On the same matrices do Gaussian elimination with partial pivoting ($[L, U] = \text{lufact}(A);$) and calculate the same quantities. Notice that partial pivoting decreases the backward error and the norms of L and U , but the performance of Gaussian elimination without pivoting is usually not conspicuously bad. That is, usually Gaussian elimination without pivoting is able to calculate the LU decomposition (of a random matrix) more or less stably.
- (b) To demonstrate the weakness of Gaussian elimination without pivoting, give it a matrix for which at least one of the pivots is guaranteed to be small. The easiest way to do this is to use matrices whose $(1, 1)$ entry is tiny. Repeat the experiments from part (a) using matrices for which a_{11} is tiny. For example, take $A = \text{randn}(n); A(1, 1) = 50 \cdot \text{eps} \cdot A(1, 1);$. MATLAB's *machine epsilon* eps equals $2u$, twice the unit roundoff. For these experiments n need not be large. Try several choices of n , but $n = 2$ is already big enough.

□

Exercise 2.7.26 Let A_n denote the $n \times n$ matrix whose form is illustrated by

$$A_5 = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 \\ -1 & 1 & 0 & 0 & 1 \\ -1 & -1 & 1 & 0 & 1 \\ -1 & -1 & -1 & 1 & 1 \\ -1 & -1 & -1 & -1 & 1 \end{bmatrix}.$$

Show that if Gaussian elimination with partial pivoting is used, then A_n can be reduced to upper-triangular form without row interchanges, and the resulting matrix U has $u_{nn} = 2^{n-1}$. Thus $\|U\|_\infty / \|A\|_\infty = 2^{n-1}/n$. □

Exercise 2.7.27 It is pointed out in [71] that the LU decomposition of a random matrix is anything but random. If $A = LU$, then $U = L^{-1}A$. We have stability if $\|U\|$ is not too much larger than $\|A\|$, and this will be the case if $\|L^{-1}\|$ is not large. In other words, a necessary condition for instability is that $\|L^{-1}\|$ be large. In this exercise we will see by experiment that Gaussian elimination with partial pivoting tends to return matrices for which L^{-1} is not large.

- (a) Write a MATLAB program that generates random unit lower-triangular matrices with entries between 1 and -1. For example, you can start with the identity

matrix ($L = \text{eye}(n)$) and then fill in the lower triangular part. The command $L(i, j) = 2 * \text{rand} - 1$ gives a random number uniformly distributed in $[-1, 1]$. Calculate the norm of L^{-1} for several such matrices with $n = 40$, 80, and 160.

- (b) Now generate the LU factors of random $n \times n$ matrices by

```
A = randn(n);
[L, U, P] = lu(A);
```

MATLAB's `lu` command does Gaussian elimination with partial pivoting. This way of using it produces a truly unit lower-triangular matrix L ; the row interchanges are incorporated into the permutation matrix P . Calculate $\|L^{-1}\|$ for several matrices generated in this way with $n = 40$, 80, and 160. Contrast your results with those of part (a).

- (c) Extend your code from part (b) so that it computes not only $\|L^{-1}\|$ but also $\|M^{-1}\|$, where M is generated from L by reversing the signs of the pivots: $M = 2I - L$.

□

Exercise 2.7.28 Show that if $L \in \mathbb{R}^{n \times n}$ and $U \in \mathbb{R}^{n \times n}$ are full lower and upper triangular, respectively, the flop count for computing the product LU is $\frac{2}{3}n^3$. □

Exercise 2.7.29 Here we prove the backward stability of Cholesky's method in detail. Let A be a positive definite matrix, and let $\hat{R}$ denote its Cholesky factor computed by some variant of Cholesky's method in floating-point arithmetic. Assume that square roots are calculated accurately: $\text{fl}(\sqrt{s}) = \sqrt{s}(1 + \epsilon)$, where $|\epsilon| \leq u$.

- (a) Using the proof of Theorem 2.7.13 as a model, prove that $A + E = \hat{R}^T \hat{R}$, where

$$|E| \leq 2nu |\hat{R}|^T |\hat{R}| + O(u^2)$$

and

$$\|E\|_F \leq 2nu \|\hat{R}\|_F^2 + O(u^2).$$

- (b) The *trace* of a matrix $B \in \mathbb{R}^{n \times n}$ is $\text{tr}(B) = \sum_{i=1}^n b_{ii}$. Use the Cauchy-Schwarz inequality to prove that $|\text{tr}(B)| \leq \sqrt{n} \|B\|_F$. (Notice that equality is attained when $B = I$. More commonly $|\text{tr}(B)| \approx \|B\|_F$.)
- (c) Prove that if $A + E = \hat{R}^T \hat{R}$, then $\|\hat{R}\|_F^2 = \text{tr}(A + E) = \text{tr}(A) + \text{tr}(E)$. (This holds regardless of whether or not $\hat{R}$ is triangular.) Thus $\|\hat{R}\|_F^2 \leq \sqrt{n} (\|A\|_F + \|E\|_F)$.
- (d) Substituting this last inequality into the result of part (a), show that

$$\|E\|_F \leq 2n^{3/2}u (\|A\|_F + \|E\|_F) + O(u^2)$$

and, if $2n^{3/2}u < 1$,

$$\|E\|_F \leq \frac{2n^{3/2}u}{1 - 2n^{3/2}u} \|A\|_F + O(u^2).$$

For realistic values of n and u we generally have $2n^{3/2}u \ll 1$, so the denominator in this expression is about 1. (Consider, e.g., the huge value $n = 10^6$ and IEEE double precision's $u \approx 10^{-16}$.) Furthermore, the factor $2n^{3/2}$ in the numerator is based on a pessimistic worst-case analysis and can be ignored. Thus we have, in practice,

$$\|E\|_F \approx Cu\|A\|_F,$$

where C is a modest multiple of 1, not $n^{3/2}$. Thus Cholesky's method for positive definite matrices is stable. □

2.8 SCALING

In a linear system $Ax = b$, any equation can be multiplied by any nonzero constant without changing the solution of the system. Such an operation is called a *row scaling* operation. A similar operation can be applied to a column of A . In contrast to row scaling operations, *column scaling* operations do change the solution.

Exercise 2.8.1 Show that if the nonsingular linear system $Ax = b$ is altered by multiplication of its j th column by $c \neq 0$, then the solution is altered only in the j th component, which is multiplied by $1/c$. □

Scaling operations can be viewed as changes of measurement units. Suppose the entries of the j th column of A are masses expressed in grams, and x_j is an acceleration measured in meters/sec². Multiplication of the j th column by $1/1000$ is the same as changing the units of its entries from grams to kilograms. At the same time x_j is multiplied by 1000, which is the same as changing its units from meters/second² to millimeters/second².

A discussion of scaling operations is made necessary by the fact that these operations affect the numerical properties of the system. This discussion has been placed near the end of the chapter because in most cases rescaling is unnecessary and undesirable; usually an appropriate scaling is determined by the physical units of the problem. Consider for example the electrical circuit problem in Example 1.2.6. In the linear system derived there, all entries of the coefficient matrix have the same units (1/ohm), all components of the solution have the same units (volts), and all components of the right-hand side have the same units (amperes). One could rescale this system so that, for example, one of the unknowns is expressed in millivolts while the others remain in volts, but this should not be done without a good reason. In most cases it is best not to rescale.

Let us look at some examples that illustrate some of the effects of scaling.

Example 2.8.2 The first example shows that a small pivot cannot be “cured” by multiplying its row by a large number. Consider the system

$$\begin{bmatrix} 2.000 & 1231. & 2471. \\ 1.196 & 3.165 & 2.543 \\ 1.475 & 4.271 & 2.142 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 3704. \\ 6.904 \\ 7.888 \end{bmatrix},$$

which was obtained from the system (2.6.10) of Example 2.6.9 by multiplying the first row by 1000. We used (2.6.10) to illustrate the damaging effects of using a small number as a pivot. Now that the first row has been multiplied by 1000, the (1, 1) entry is no longer small. It is now the largest entry in the first column. Let us see what happens when it is used as a pivot.

Using four-digit decimal arithmetic, the multipliers for the first step are

$$l_{21} = \frac{1.196}{2.000} = .5980 \quad \text{and} \quad l_{31} = \frac{1.475}{2.000} = .7375.$$

Comparing these with (2.6.11), we see that they are 1000 times smaller than before. In step 1 the (2, 2) entry is altered as follows:

$$3.165 - (.5980)(1231.) = 3.165 - 736.1 = -732.9.$$

Comparing this with (2.6.12), we see that in spite of the smaller pivot, the outcome is the same as before. This time the number 3.165 is swamped by the large entry 1231. Notice that this computation is essentially identical with (2.6.12). The result is *exactly* the same, including the roundoff errors. You can check that when the (2, 3), (3, 2), and (3, 3) entries are modified, swamping occurs just as before, and indeed the computations are essentially the same as before and yield exactly the same result. Thus, after the first step, the modified coefficient matrix is

$$\begin{bmatrix} 2.000 & 1231. & 2471. \\ .5980 & -732.9 & -1475. \\ .7375 & -903.6 & -1820. \end{bmatrix}.$$

The submatrix for the second step is

$$\begin{bmatrix} -732.9 & -1475. \\ -903.6 & -1820. \end{bmatrix},$$

which is exactly the same as before. If we continue the computation, we will have the same disastrous outcome. This time swamping occurred not because large multiples of the first row were subtracted from the other rows, but because the first row itself is large.

How could this disaster have been predicted? Looking at the coefficient matrix, we can see that it is ill conditioned: the rows (and the columns) are out of scale. It is interesting that we have two different explanations for the same disaster. With

the original system we blamed a small pivot; with the rescaled system we blame ill conditioning. $\square$

This example illustrated an interesting theorem of F. L. Bauer. Suppose we solve the system $Ax = b$ by Gaussian elimination, using some specified sequence of row and column interchanges, on a computer that uses base β floating-point arithmetic. If the system is then rescaled by multiplying the rows and columns by powers of β and solved again using the same sequence of row and column interchanges, the result will be exactly the same as before, including roundoff errors. All roundoff errors at all steps are the same as before. In our present example $\beta = 10$; multiplying the first row by 10^3 has no effect on the arithmetic. It is not hard to prove Bauer's theorem; you might like to do so as an exercise or at least convince yourself that it is true. The examples that follow should help.

Bauer's theorem has an interesting consequence. If the scaling factors are always chosen to be powers of β , then the only way rescaling affects the numerical properties of Gaussian elimination is by changing the choices of pivot. If scaling factors that are not powers of β are used, there will be additional roundoff errors associated with the rescaling, but it remains true that the principal effect of rescaling is to alter the pivot choices.

Example 2.8.3 Let us solve the system

$$\begin{bmatrix} .003 & .217 \\ .277 & .138 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} .437 \\ .553 \end{bmatrix} \quad (2.8.4)$$

using three-digit decimal floating-point arithmetic without row or column interchanges. The exact solution is $x = [1, 2]^T$. The multiplier is $l_{21} = .277/.003 = 92.3$, and $u_{22} = .138 - (92.3)(.217) = .138 - 20.0 = -19.9$, so the computed LU decomposition is

$$\begin{bmatrix} 1 & 0 \\ 92.3 & 1 \end{bmatrix} \begin{bmatrix} .003 & .217 \\ 0 & -19.9 \end{bmatrix}.$$

The forward substitution gives $y_1 = .437$ and $y_2 = .553 - (92.3)(.437) = .553 - 40.3 = -39.7$. Finally the back substitution gives $x_2 = (-39.7)/(-19.9) = 1.99$ and $x_1 = .437 - (.217)(1.99)/(.003) = (.437 - .432)/(.003) = (.005)/(.003) = 1.67$. Thus the computed solution is $\hat{x} = [1.67, 1.99]^T$, whose first component is inaccurate. $\square$

Exercise 2.8.5

- Calculate $\kappa_\infty(A)$, where A is the coefficient matrix of (2.8.4). Observe that A is well conditioned.
- Perform Gaussian elimination on (2.8.4) with the rows interchanged, using three-digit decimal floating-point arithmetic, and note that an accurate solution is obtained. (Remember to round each intermediate result to three decimal places before using it in the next calculation.)

□

Example 2.8.6 Now let us solve

$$\begin{bmatrix} .300 & 21.7 \\ .277 & .138 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 43.7 \\ .553 \end{bmatrix}, \quad (2.8.7)$$

which was obtained by multiplying the first row of (2.8.4) by 10^2 . Now the (1, 1) entry is the largest entry in the first column. Again we use three-digit decimal arithmetic and no row or column interchanges. By Bauer's theorem the outcome should be the same as in Example 2.8.3. Let us check that it is. The multiplier is $l_{21} = .277/.300 = .923$, and $u_{22} = .138 - (.923)(21.7) = .138 - 20.0 = -19.9$, so the computed LU decomposition is

$$\begin{bmatrix} 1 & 0 \\ .923 & 1 \end{bmatrix} \begin{bmatrix} .300 & 21.7 \\ 0 & -19.9 \end{bmatrix}.$$

The forward substitution gives $y_1 = 43.7$ and $y_2 = .553 - (.923)(43.7) = .553 - 40.3 = -39.7$. Finally the back substitution yields $x_2 = (-39.7)/(-19.9) = 1.99$ and $x_1 = 43.7 - (21.7)(1.99)/(.300) = (43.7 - 43.2)/(.300) = (.500)/(.300) = 1.67$. Thus the computed solution is again $\hat{x} = [1.67, 1.99]^T$. All intermediate results are identical to those in Example 2.8.3, except for powers of 10. □

Exercise 2.8.8

- Calculate $\kappa_\infty(A)$, where A is the coefficient matrix of (2.8.7). A is ill conditioned (relative to three-digit decimal arithmetic) because its rows (and columns) are out of scale.
- Perform Gaussian elimination on (2.8.7) with the rows interchanged, using three-digit decimal arithmetic. Note that, as guaranteed by Bauer's theorem, the computations and outcome are identical to those in part (b) of Exercise 2.8.5. Thus an ill-conditioned coefficient matrix does not absolutely guarantee an inaccurate result. (However, if the partial-pivoting strategy had been used, the row interchange would not have been made, and the outcome would have been bad.)

□

Exercise 2.8.9 Solve (2.8.7) by Gaussian elimination with the columns interchanged, using three-digit decimal arithmetic. This is the complete pivoting strategy. Note that a good result is obtained. □

Example 2.8.10 Now let us solve

$$\begin{bmatrix} .300 & .217 \\ .277 & .00138 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 43.7 \\ .553 \end{bmatrix}, \quad (2.8.11)$$

which was obtained from (2.8.7) by multiplying the second column by $1/100$. The exact solution is therefore $x = [1, 200]^T$. The (1, 1) entry is now the largest

entry in the matrix. Again we use three-digit decimal arithmetic and no row or column interchanges (which is the choice that both the partial and complete pivoting strategies would make). By Bauer's theorem the outcome should be the same as in Examples 2.8.3 and 2.8.6. The multiplier is $l_{21} = .277/.300 = .923$, and $u_{22} = .00138 - (.923)(.217) = .00138 - .200 = -.199$, so the computed LU decomposition is

$$\begin{bmatrix} 1 & 0 \\ .923 & 1 \end{bmatrix} \begin{bmatrix} .300 & .217 \\ 0 & -.199 \end{bmatrix}.$$

The forward substitution gives $y_1 = 43.7$ and $y_2 = .553 - (.923)(43.7) = .553 - 40.3 = -39.7$. Finally the back substitution yields $x_2 = (-39.7)/(-.199) = 199.$, and $x_1 = 43.7 - (.217)(199.)/(.300) = (43.7 - 43.2)/(.300) = (.500)/(.300) = 1.67$. Thus the computed solution is $\hat{x} = [1.67, 199.]^T$. All computations were identical to those in Examples 2.8.3 and 2.8.6.

Although the computed solution $\hat{x} = [1.67, 199.]^T$ has an inaccurate first component, it should not necessarily be viewed as a bad result. The inaccurate entry is much smaller than the accurate one, and in fact $\|\delta x\|_\infty/\|x\|_\infty = .005$, where $\delta x = \hat{x} - x$. This is an excellent outcome for three-digit decimal arithmetic. The small value of $\|\delta x\|_\infty/\|x\|_\infty$ is guaranteed by the well-conditioned coefficient matrix, together with the fact that the computed $\hat{L}$ and $\hat{U}$ do not have large entries (cf. Theorems 2.7.14 and 2.3.6).

It is easy to imagine situations in which the computed result $\hat{x} = [1.67, 199.]^T$ is acceptable. Suppose for example that x_1 and x_2 represent voltages expressed in the same units. If all that matters is the voltage difference, then the result is okay, since the computed difference $\hat{x}_2 - \hat{x}_1 = 197.33$ differs from the correct difference 199 by only about one percent. $\square$

Exercise 2.8.12

- Calculate $\kappa_\infty(A)$, where A is the coefficient matrix of (2.8.11).
- Perform Gaussian elimination on (2.8.11) with the rows interchanged, using three-digit decimal arithmetic.
- Perform Gaussian elimination on (2.8.11) with the columns interchanged, using three-digit decimal arithmetic.

$\square$

2.9 COMPONENTWISE SENSITIVITY ANALYSIS

In this chapter we have taken the oldest and simplest approach to sensitivity analysis, in which everything is measured by norms. It is called *normwise sensitivity analysis*, and it is accompanied by *normwise backward error analysis*. This style of error analysis has been very successful, but there are some situations in which a different type of analysis, *componentwise sensitivity analysis*, is more appropriate. In the

normwise analysis, δA is considered a small perturbation of A if $\|\delta A\|/\|A\|$ is small. This criterion does not force all of the ratios $|\delta a_{ij}|/|a_{ij}|$, which are the perturbations of the components, to be small.

Example 2.9.1 Suppose

$$A = \begin{bmatrix} 1.04 & 2.35 \\ 4.26 \times 10^{-6} & 6.32 \end{bmatrix} \quad \text{and} \quad \delta A = \begin{bmatrix} 1.32 \times 10^{-5} & 5.46 \times 10^{-6} \\ 1.02 \times 10^{-5} & 8.29 \times 10^{-6} \end{bmatrix}.$$

Then

$$A + \delta A = \begin{bmatrix} 1.0400132 & 2.35000546 \\ 1.446 \times 10^{-5} & 6.32000829 \end{bmatrix}.$$

We have $\|\delta A\|_\infty/\|A\|_\infty < 10^{-5}$, but this does not force $|\delta a_{21}|/|a_{21}|$ to be small. Obviously this can happen to any entry for which $|a_{ij}| \ll \|A\|$. $\square$

In componentwise sensitivity analysis, perturbations are considered small only if the perturbation in each component is small relative to that component, that is,

$$\max_{i,j} \frac{|\delta a_{ij}|}{|a_{ij}|}$$

is small. Since we often encounter matrices with zero entries, we prefer the following reformulation, in which a_{ij} does not appear in the denominator: The perturbation δA is *componentwise ϵ -small* with respect to A if there is a positive $\epsilon \ll 1$ such that

$$|\delta a_{ij}| \leq \epsilon |a_{ij}| \quad \text{for } i, j = 1, \dots, n. \quad (2.9.2)$$

Notice that under this condition, if $a_{ij} = 0$, then $\delta a_{ij} = 0$. Thus sparse matrices stay sparse and sparsity patterns are preserved under perturbations of this type.

Recall the following notation, which we introduced in Section 2.7. If B is a matrix (or vector) with (i, j) entry b_{ij} , then $|B|$ is the matrix with the same dimensions whose (i, j) entry is $|b_{ij}|$. We write $|B| \leq |C|$ to mean that $|b_{ij}| \leq |c_{ij}|$ for all i and j . With these notational conventions we can rewrite the condition (2.9.2) as

$$|\delta A| \leq \epsilon |A|. \quad (2.9.3)$$

In the componentwise sensitivity analysis we can ask the same sort of questions as we do in the normwise analysis. For example, if $Ax = b$ and $(A + \delta A)(x + \delta x) = b$, what is the largest δx can be relative to x ? Before considering this and related questions, we pause to establish some basic facts about the matrix absolute value notation and matrix inequalities.

Exercise 2.9.4

- Show that if $A = BC$, then $|A| \leq |B||C|$. (This is a matrix inequality.) In particular, $|Ax| \leq |A||x|$.
- (Review) Show that if $y = |x|$, then $\|y\|_\infty = \|x\|_\infty$.

$\square$

If $A \in \mathbb{R}^{n \times n}$ is nonsingular, we can build various other $n \times n$ matrices from A , for example, $|A|$, A^{-1} , $|A^{-1}|$, and $K = |A^{-1}| |A|$. This last one appears in various error bounds, as we shall see, and we use it to define a new type of condition number, the *Skeel condition number* of A :

$$\text{skeel}(A) = \|K\|_{\infty} = \||A^{-1}||A|\|_{\infty}.$$

Now we are ready to prove some theorems. The first is the componentwise analogue of Theorem 2.2.4

Theorem 2.9.5 *Let $A \in \mathbb{R}^{n \times n}$ be nonsingular, let $b \in \mathbb{R}^n$ be nonzero, and let x be the unique solution of $Ax = b$. Suppose $\hat{x} = x + \delta x$ is the solution of $A\hat{x} = b + \delta b$, where*

$$|\delta b| \leq \epsilon |b|.$$

Then

$$|\delta x| \leq \epsilon |A^{-1}| |A| |x| \quad (2.9.6)$$

and

$$\frac{\|\delta x\|_{\infty}}{\|x\|_{\infty}} \leq \epsilon \text{skeel}(A). \quad (2.9.7)$$

Exercise 2.9.8 Prove Theorem 2.9.5 as follows (or do it your own way).

- (a) Prove that $|\delta x| \leq |A^{-1}| |\delta b|$ and $|b| \leq |A| |x|$. Then deduce (2.9.6).
- (b) Deduce (2.9.7) from (2.9.6).

□

Now consider this componentwise analogue of Theorems 2.3.3 and 2.3.6.

Theorem 2.9.9 *Let $A \in \mathbb{R}^{n \times n}$ be nonsingular, let $b \in \mathbb{R}^n$ be nonzero, and let x be the unique solution of $Ax = b$. Suppose $\hat{x} = x + \delta x$ satisfies $(A + \delta A)\hat{x} = b$, where*

$$|\delta A| \leq \epsilon |A|.$$

Then

$$|\delta x| \leq \epsilon |A^{-1}| |A| |\hat{x}| \quad (2.9.10)$$

and

$$\frac{\|\delta x\|_{\infty}}{\|\hat{x}\|_{\infty}} \leq \epsilon \text{skeel}(A). \quad (2.9.11)$$

If $\epsilon \text{skeel}(A) < 1$, then also

$$\frac{\|\delta x\|_{\infty}}{\|x\|_{\infty}} \leq \frac{\epsilon \text{skeel}(A)}{1 - \epsilon \text{skeel}(A)}. \quad (2.9.12)$$

Exercise 2.9.13 Prove Theorem 2.9.9 as follows (or do it your own way).

- (a) Prove that $\delta x = -A^{-1}\delta A\hat{x}$ and $|\delta x| \leq |A^{-1}| |\delta A| |\hat{x}|$. Then deduce (2.9.10).
- (b) Deduce (2.9.11) from (2.9.10).
- (c) Multiply (2.9.11) through by $\|\hat{x}\|_\infty$, apply the triangle inequality to break $\|\hat{x}\|_\infty$ into two parts, and deduce (2.9.12), remembering to point out where you are using the added hypothesis $\epsilon \text{ skeel}(A) < 1$.

□

Componentwise Backward Error Analysis

Componentwise sensitivity analysis can be combined with componentwise backward stability analysis, whenever the latter is successful. One example of componentwise backward stability that we have already encountered is the forward substitution algorithm for solving triangular systems. Theorem 2.7.2 states that if we solve the triangular system $Gy = b$ by forward substitution using floating-point arithmetic, then the computed solution $\hat{y}$ satisfies $(G + \delta G)\hat{y} = b$, where $|\delta G| \leq 2nu|G| + O(u^2)$. This is a componentwise backward stability result, since it says that each component of δG is tiny relative to the corresponding component of G . As a practical matter we have $|\delta G| \leq Cu|G|$, where C is a modest multiple of 1. We can now apply Theorem 2.9.9 with $\epsilon = Cu$, to conclude that $\hat{y}$ is accurate (in the sense that $\|\delta y\|_\infty / \|\hat{y}\|_\infty$ is tiny) if the Skeel condition number $\text{skeel}(G)$ is not too large.

Iterative Refinement

Another process that yields componentwise backward stability is iterative refinement. This is an old procedure that was originally used to improve the accuracy of solutions to ill-conditioned systems. Let $\hat{x}$ denote an approximation to the solution of the system $Ax = b$, and let $\hat{r}$ be the associated residual: $\hat{r} = b - A\hat{x}$. The approximation $\hat{x}$ may have been obtained by Gaussian elimination, for example. If we could solve the residual system $Az = \hat{r}$ exactly, then the vector $x = \hat{x} + z$ would be the exact solution of $Ax = b$, as you can easily check. If we did obtain $\hat{x}$ by Gaussian elimination, then an LU decomposition is available, so we can solve $Az = \hat{r}$ inexpensively. Of course the computed solution $\hat{z}$ is not exact. If A is somewhat ill conditioned, it may be far from exact. Nevertheless it is not unreasonable to hope that $\hat{x} + \hat{z}$ will be an improvement over $\hat{x}$. If this is the case, then perhaps we can improve the solution even more by calculating the residual associated with $\hat{x} + \hat{z}$ and repeating the process. In fact we can repeat it as many times as we wish. This gives the following *iterative refinement* algorithm.

Iterative Refinement Algorithm

for $k = 1, \dots, m$

$\hat{r} \leftarrow b - A\hat{x}$ Calculate $\hat{z}$, an approximate solution of $Az = \hat{r}$. (Use the <i>LU</i> decomposition that was computed previously.) $\hat{x} \leftarrow \hat{x} + \hat{z}$ if ($\ \hat{z}\ /\ \hat{x}\ $ is sufficiently small) exit (successful completion)	(2.9.14)
---	----------

Set flag indicating failure.

This is called an *iterative* algorithm because the number of steps or *iterations* to be performed is not known in advance. The iterations are terminated as soon as the corrections become sufficiently small. Any iterative algorithm should have an upper bound on the number of iterations it is willing to attempt before abandoning the process as a failure. In (2.9.14) that number is denoted by m . Notice that in order to carry out this procedure, we must save copies of A and b to use in the computation of the residuals.

Up until about 1980 it was thought that (2.9.14) cannot hope to succeed unless the residuals are calculated in extended-precision arithmetic. This means that if we are using single-precision arithmetic, the step $\hat{r} \leftarrow b - A\hat{x}$ should be done in double precision. The reason for this is that severe cancellation occurs when $A\hat{x}$ is subtracted from b . The smaller $\hat{r}$ becomes, the worse the cancellation is. The objective of the extended-precision computation is to preserve as many significant digits as possible in the face of this cancellation.

If iterative refinement is carried out in this way and the system is not too badly conditioned (say $\kappa(A) \ll 1/u$), (2.9.14) actually does converge to the true solution of $Ax = b$. Thus iterative refinement can be used to solve the problem to full precision ($\|x - \hat{x}\| \approx u$). There is a catch, however. The system that is solved so precisely is the one whose coefficient matrix A and right-hand-side vector b are exactly what is stored in the computer. Because of measurement and representation errors, the stored A and b are mere approximations to the true data for the physical problem that we are trying to solve. If the problem is ill conditioned, the exact solution of $Ax = b$ can be a very unsatisfactory approximation to the solution of the problem we really wish to solve.

Around 1980 it was realized that iterative refinement is useful even if the residuals are not computed with extended precision arithmetic. Full precision ($\|x - \hat{x}\| \approx u$) cannot be attained, but some improvement in accuracy is possible. Furthermore, iterative refinement has a good effect on the backward error. Skeel [61] showed that if the system is not too badly conditioned and not too badly out of scale, then *one step* of iterative refinement is usually enough to ensure a *componentwise* backward stable solution. See [41] or [61] for details.

Componentwise backward stability means that the computed solution satisfies $(A + \delta A)\hat{x} = b$, where $|\delta A| \leq Cu|A|$, with C not too big. This can then be

combined with Theorem 2.9.9 to get the bound

$$\frac{\|\delta x\|_{\infty}}{\|\hat{x}\|_{\infty}} \leq C u \text{skel}(A).$$

This is sometimes significantly better than bounds that can be obtained using the normwise error analysis, because $\text{skel}(A)$ can be much smaller than $\kappa_{\infty}(A)$ (See Exercise 2.9.15). The greatest advantage comes with matrices that are ill-conditioned simply because the rows are out of scale. The Skeel condition number $\text{skel}(A)$ is insensitive to row scaling, so it remains small while the normwise condition number $\kappa_{\infty}(A)$ becomes large in proportion to the badness of the scaling.

There are inexpensive methods for estimating $\text{skel}(A)$ that work on the same principal as condition estimators for $\kappa_1(A)$. See [41].

Exercise 2.9.15

- Show that for every nonsingular matrix A , $\text{skel}(A) \leq \kappa_{\infty}(A)$.
- Show that if D is a nonsingular diagonal matrix, then $|DA| = |D| |A|$ and $|D|^{-1} = |D^{-1}|$.
- Show that the Skeel condition number is invariant under row scaling; that is, $\text{skel}(DA) = \text{skel}(A)$ for all nonsingular diagonal matrices D .
- From part (a) we know that the ratio $\kappa_{\infty}(A)/\text{skel}(A)$ is always at least one. Show by example that it can be made arbitrarily large.

□

Exercise 2.9.16

- Prove that if A is nonsingular, $A + \delta A$ is singular, and $|\delta A| \leq \epsilon |A|$, then $\epsilon \text{skel}(A) \geq 1$ (cf. Theorem 2.3.1).
- Prove that if A is nonsingular and $|\delta A| \leq \epsilon |A|$, where $\epsilon \text{skel}(A) < 1$, then $A + \delta A$ is nonsingular.
- Discuss the relationship between the result in part (b) and Theorem 2.3.1. Is one stronger than the other?

□

3

The Least Squares Problem

In this chapter we study the least squares problem, which arises repeatedly in scientific and engineering computations. After describing the problem in Section 3.1, we immediately begin to develop the tools that we need to solve the problem: rotators, reflectors, and the QR decomposition, in Section 3.2. In Section 3.3 we show how to use a QR decomposition to solve the least squares problem. In Section 3.4 we introduce the Gram-Schmidt orthonormalization process, demonstrate its relationship to the QR decomposition, and study some of its computational variants. In the interest of getting to the algorithms quickly, we have postponed discussion of some important theoretical issues to Section 3.5. Section 3.6 addresses the important question of updating the QR decomposition when a row or column is added to or deleted from the data matrix.

3.1 THE DISCRETE LEAST SQUARES PROBLEM

A task that occurs frequently in scientific investigations is that of finding a straight line that “fits” some set of data points. Typically we have a fairly large number of points (t_i, y_i) , $i = 1, \dots, n$, collected from some experiment, and often we have some theoretical reason to believe that these points should lie on a straight line. Thus we seek a linear function $p(t) = a_0 + a_1 t$ such that $p(t_i) = y_i$, $i = 1, \dots, n$. In practice of course the points will deviate from a straight line, so it is impossible to find a linear $p(t)$ that passes through all of them. Instead we settle for a line that fits the data well, in the sense that the errors

$$|y_i - p(t_i)| \quad i = 1, \dots, n$$

are made as small as possible.

It is generally impossible to find a p for which all of the numbers $|y_i - p(t_i)|$ are simultaneously minimized, so instead we seek a p that strikes a good compromise. Specifically, let $r = [r_1, \dots, r_n]^T$ denote the vector of residuals $r_i = y_i - p(t_i)$. We can solve our problem by choosing a vector norm $\|\cdot\|$ and taking our compromise function to be that p for which $\|r\|$ is made as small as possible. Of course the solution depends on the choice of norm. For example, if we choose the Euclidean norm, we minimize the quantity

$$\|r\|_2 = \left(\sum_{i=1}^n |y_i - p(t_i)|^2 \right)^{1/2},$$

whereas if we choose the 1-norm or the ∞ -norm, we minimize respectively the quantities

$$\|r\|_1 = \sum_{i=1}^n |y_i - p(t_i)| \quad \text{and} \quad \|r\|_\infty = \max_{1 \leq i \leq n} |y_i - p(t_i)|.$$

The problem of finding the minimizing p has been studied for a variety of norms, including those we have just mentioned. By far the nicest theory is that based on the Euclidean norm, and it is that theory that we will study in this chapter. To minimize $\|r\|_2$ is the same as to minimize

$$\|r\|_2^2 = \sum_{i=1}^n |y_i - p(t_i)|^2.$$

Thus we are minimizing the sum of the squares of the residuals. For this reason the problem of minimizing $\|r\|_2$ is called the *least squares problem*.¹

The choice of the 2-norm can be justified on statistical grounds. Suppose the data fail to lie on a straight line because of errors in the measured y_i . If the errors are independent and normally distributed with mean zero and variance σ^2 , then the solution of the least squares problem is the maximum likelihood estimator of the true solution.

A straight line is the graph of a polynomial of first degree. Sometimes it is desirable to fit a data set with a polynomial of higher degree. For example, it might be useful to approximate the data of Figure 3.1 by a polynomial of degree 2. If we decide to approximate our data by a polynomial of degree $\leq m-1$, then the task is to seek $p(t) = a_0 + a_1 t + a_2 t^2 + \dots + a_{m-1} t^{m-1}$ such that

$$p(t_i) = y_i \quad i = 1, \dots, n. \quad (3.1.1)$$

¹It is called the *discrete* least squares problem because a finite (discrete) data set (t_i, y_i) is being approximated. The *continuous* least squares problem, in which a continuum of data points is approximated, will be discussed briefly in Section 3.5.

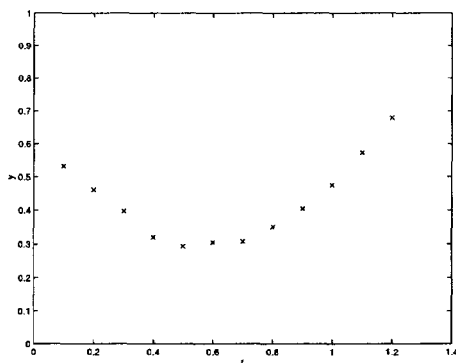


Fig. 3.1 Data that can be approximated well by a quadratic polynomial

Since the number of data points is typically large and the degree of the polynomial fairly low, it will usually be the case that $n \gg m$. In this case it is too much to ask for a p that satisfies (3.1.1) exactly, but for the moment let us act as if that were our goal. The set of polynomials of degree $\leq m-1$ is a vector space of dimension m . If $\phi_1, \phi_2, \dots, \phi_m$ is a basis of this space, then each polynomial p in the space can be expressed in the form

$$p(t) = \sum_{j=1}^m x_j \phi_j(t) \quad (3.1.2)$$

for some unique choice of coefficients $x_1, x_2, \dots, x_m$. The obvious basis is $\phi_1(t) = 1, \phi_2(t) = t, \phi_3(t) = t^2, \dots, \phi_m(t) = t^{m-1}$, but there are many others, some of which may be better from a computational standpoint. See Example 4.4.15.

Substituting the expression (3.1.2) into the equations (3.1.1), we obtain a set of n linear equations in the m unknowns $x_1, \dots, x_m$:

$$\sum_{j=1}^m x_j \phi_j(t_i) = y_i, \quad i = 1, \dots, n,$$

which can be written in matrix form as

$$\begin{bmatrix} \phi_1(t_1) & \phi_2(t_1) & \cdots & \phi_m(t_1) \\ \phi_1(t_2) & \phi_2(t_2) & \cdots & \phi_m(t_2) \\ \phi_1(t_3) & \phi_2(t_3) & \cdots & \phi_m(t_3) \\ \vdots & \vdots & \cdots & \vdots \\ \phi_1(t_n) & \phi_2(t_n) & \cdots & \phi_m(t_n) \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_m \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_n \end{bmatrix}. \quad (3.1.3)$$

If $n > m$, as is usually the case, this is an *overdetermined system*; that is, it has more equations than unknowns. Thus we cannot expect to find an x that satisfies (3.1.3) exactly. Instead we might seek an x for which the sum of the squares of the residuals is minimized.

It is easy to imagine further generalizations of this problem. For example, the functions $\phi_1, \dots, \phi_m$ could be taken to be trigonometric or exponential or some other kind of functions. More generally we can consider the overdetermined system

$$Ax = b, \quad (3.1.4)$$

where $A \in \mathbb{R}^{n \times m}$, $n > m$, and $b \in \mathbb{R}^n$. The *least squares problem* for the system (3.1.4) is to find $x \in \mathbb{R}^m$ for which $\|r\|_2$ is minimized, where $r = b - Ax$ is the vector of residuals. With the help of the orthogonal matrices introduced in the next section, we will develop algorithms for solving the least squares problem (3.1.4), which includes (3.1.3) as a special case.

Exercises

Exercise 3.1.5 Consider the following data.

t_i	1.0	1.5	2.0	2.5	3.0
y_i	1.1	1.2	1.3	1.3	1.4

- Set up an overdetermined system of the form (3.1.3) for a straight line passing through the data points. Use the standard basis polynomials $\phi_1(t) = 1$, $\phi_2(t) = t$.
- Use MATLAB to calculate the least-squares solution of the system from part (a). This is a simple matter. Given an overdetermined system $Ax = b$, the MATLAB command `x = A\b` computes the least squares solution. Recall that this is exactly the same command as would be used to tell MATLAB to solve a square system $Ax = b$ by Gaussian elimination.² Some useful MATLAB commands:

```
t = 1:.5:3; t = t'; s = ones(5,1); A = [s t];
```

We already know that MATLAB uses Gaussian elimination with partial pivoting in the square case. In the next two sections you will find out what MATLAB does in the overdetermined case.

- Use the MATLAB `plot` command to plot the five data points and your least squares straight line. Type `help plot` for information about using the plot command.
- Use MATLAB to compute $\|r\|_2$, the norm of the residual.

□

Exercise 3.1.6 Repeat Exercise 3.1.5, but this time compute the best least squares polynomial of degree ≤ 2 . Notice that in this case the norm of the residual is smaller. This is

²In the square case A is expected to be nonsingular. In the overdetermined case it is expected to have *full rank*. Rank will be discussed in Section 3.3 and in subsequent sections.

to be expected; the space of quadratic polynomials contains the space of linear polynomials, so the quadratic fit should be better or in any case no worse than the linear polynomial fit. $\square$

Exercise 3.1.7 Repeat Exercise 3.1.5, but this time compute the best least squares polynomial of degree ≤ 4 . The space of quartic polynomials has dimension 5. Thus we have $n = m = 5$; the system is not overdetermined. The solution interpolates the data exactly (except for roundoff errors). Plot the data points and the solution on the same set of axes. Make the spacing between points small enough that the curve appears smooth. Sample code.

```
tt = 1:.01:3;
p4 = x(1) + x(2)*tt + x(3)*tt.^2 + x(4)*tt.^3 + x(5)*tt.^4;
plot(...,tt,p4,'k-',...)
```

In the same plot include your least squares linear and quadratic polynomials from the previous two exercises. Notice that the latter are much less oscillatory than the fourth-degree interpolant is. They seem to represent the trend of the data better. $\square$

Exercise 3.1.8 Set up and solve a system of equations for the best least squares fit to the data of Exercise 3.1.5 from the space spanned by the three functions $\phi_1(t) = 1$, $\phi_2(t) = e^t$, and $\phi_3(t) = e^{-t}$. $\square$

Exercise 3.1.9 Consider the following data.

t_i	1	2	3
y_i	1.1	1.3	1.4

- Set up a system of three equations in three unknowns that could be solved to find the unique quadratic polynomial that interpolates the data. (As you may know, this is just one of several ways to solve this problem.)
- Solve the system using MATLAB to find the interpolant.
- Set up a system of three equations in three unknowns that could be solved to find the unique linear combination of the functions $\phi_1(t) = 1$, $\phi_2(t) = e^t$, and $\phi_3(t) = e^{-t}$ that interpolates the data.
- Solve the system using MATLAB to find this other interpolant.
- Plot the two interpolants on the same set of axes.

$\square$

3.2 ORTHOGONAL MATRICES, ROTATORS, AND REFLECTORS

In this section we will develop powerful tools for solving the least squares problem. As these tools will also be used heavily in the chapters that follow, this is one of the most important sections of the book.

As we did in the first two chapters, we will restrict our attention to real vectors and matrices. However, the entire theory can be carried over to the complex case, and this is done in part in exercises at the end of the section. The results of those exercises will be used in Chapter 5.

We begin by introducing an inner product in $\mathbb{R}^n$. Given two vectors $x = [x_1, \dots, x_n]^T$ and $y = [y_1, \dots, y_n]^T$ in $\mathbb{R}^n$, we define the *inner product* of x and y , denoted $\langle x, y \rangle$ by

$$\langle x, y \rangle = \sum_{i=1}^n x_i y_i.$$

Although the inner product is a real number, it can also be expressed as a matrix product:

$$\langle x, y \rangle = y^T x = x^T y.$$

The inner product has the following properties, which you can easily verify:

$$\begin{aligned} \langle x, y \rangle &= \langle y, x \rangle \\ \langle \alpha_1 x_1 + \alpha_2 x_2, y \rangle &= \alpha_1 \langle x_1, y \rangle + \alpha_2 \langle x_2, y \rangle \\ \langle x, \alpha_1 y_1 + \alpha_2 y_2 \rangle &= \alpha_1 \langle x, y_1 \rangle + \alpha_2 \langle x, y_2 \rangle \\ \langle x, x \rangle &\geq 0 \quad \text{with equality if and only if } x = 0. \end{aligned}$$

for all $x, x_1, x_2, y, y_1, y_2 \in \mathbb{R}^n$ and all $\alpha_1, \alpha_2 \in \mathbb{R}$.

Note also the close relationship between the inner product and the Euclidean norm:

$$\|x\|_2 = \sqrt{\langle x, x \rangle}.$$

The Cauchy-Schwarz inequality (Theorem 2.1.6) can be stated more concisely in terms of the inner product and the Euclidean norm: For every $x, y \in \mathbb{R}^n$,

$$|\langle x, y \rangle| \leq \|x\|_2 \|y\|_2. \quad (3.2.1)$$

When $n = 2$ (or 3) the inner product coincides with the familiar dot product from analytic geometry. Recall that if x and y are two nonzero vectors in a plane and θ is the angle between them, then

$$\cos \theta = \frac{\langle x, y \rangle}{\|x\|_2 \|y\|_2}.$$

It is not unreasonable to employ this formula to *define* the angle between two vectors in $\mathbb{R}^n$. Note first that (3.2.1) guarantees that $\langle x, y \rangle / \|x\|_2 \|y\|_2$ always lies between -1 and 1 , so it is the cosine of some angle. We now define the *angle* between (nonzero) x and $y \in \mathbb{R}^n$ to be

$$\theta = \arccos \frac{\langle x, y \rangle}{\|x\|_2 \|y\|_2}. \quad (3.2.2)$$

If $x = 0$ or $y = 0$, we define $\theta = \pi/2$ radians. Two vectors x and y are said to be *orthogonal* if the angle between them is $\pi/2$ radians. Clearly x and y are orthogonal if and only if $\langle x, y \rangle = 0$.

The angle between two vectors in, say, $\mathbb{R}^{100}$ is just as real as the angle between two vectors in a plane. In fact x and y span a two-dimensional subspace of $\mathbb{R}^{100}$. This subspace is nothing but a copy of $\mathbb{R}^2$, that is, a plane. Viewed in this plane, x and y have an angle between them. It is this angle that is produced by the formula (3.2.2).

Orthogonal Matrices

A matrix $Q \in \mathbb{R}^{n \times n}$ is said to be *orthogonal* if $QQ^T = I$. This equation says that Q has an inverse, and $Q^{-1} = Q^T$. Since a matrix always commutes with its inverse, we have $Q^TQ = I$ as well. For square matrices the equations

$$QQ^T = I \quad Q^TQ = I \quad Q^T = Q^{-1}$$

are all equivalent, and any one of them could be taken as the definition of an orthogonal matrix.

Exercise 3.2.3 (a) Show that if Q is orthogonal, then Q^{-1} is orthogonal. (b) Show that if Q_1 and Q_2 are orthogonal, then Q_1Q_2 is orthogonal. $\square$

Exercise 3.2.4 Show that if Q is orthogonal, then $\det(Q) = \pm 1$. $\square$

Exercise 3.2.5 Recall that a *permutation matrix* is a matrix that has exactly one 1 in each row and in each column, all other entries being zero. In Section 1.8 we introduced permutation matrices as a way to represent row interchanges in Gaussian elimination. Show that every permutation matrix is orthogonal. $\square$

Theorem 3.2.6 If $Q \in \mathbb{R}^{n \times n}$ is orthogonal, then for all $x, y \in \mathbb{R}^n$,

$$(a) \langle Qx, Qy \rangle = \langle x, y \rangle, \quad (b) \|Qx\|_2 = \|x\|_2.$$

Proof. (a) $\langle Qx, Qy \rangle = (Qy)^T(Qx) = y^TQ^TQx = y^TIx = y^Tx = \langle x, y \rangle$.

(b) Set $x = y$ in part (a) and take square roots. $\square$

Part (b) of the theorem says that Qx and x have the same length. Thus *orthogonal transformations preserve lengths*. Combining parts (a) and (b), we see that

$$\arccos \frac{\langle Qx, Qy \rangle}{\|Qx\|_2 \|Qy\|_2} = \arccos \frac{\langle x, y \rangle}{\|x\|_2 \|y\|_2}.$$

Thus the angle between Qx and Qy is the same as the angle between x and y . We conclude that *orthogonal transformations preserve angles*.

In the least squares problem for an overdetermined system we wish to find x such that $\|b - Ax\|_2$ is minimized. Theorem 3.2.6, part (b), shows that for every orthogonal matrix Q , $\|b - Ax\|_2 = \|Qb - QA x\|_2$. Therefore the solution of the least squares problem is unchanged when A and b are replaced by QA and Qb , respectively. We will eventually solve the least squares problem by finding a Q for

which QA has a very simple form, from which the solution of the least squares problem will be easy to determine.

Exercise 3.2.7 Prove the converse of part (a) of Theorem 3.2.6: If Q satisfies $\langle Qx, Qy \rangle = \langle x, y \rangle$ for all x and y in $\mathbb{R}^n$, then Q is orthogonal. (The condition of part (b) also implies that Q is orthogonal, but this is harder to prove.) $\square$

Exercise 3.2.8 Show that if Q is orthogonal, then $\|Q\|_2 = 1$, $\|Q^{-1}\|_2 = 1$, and $\kappa_2(Q) = 1$. Thus Q is perfectly conditioned with respect to the 2-condition number. This suggests that orthogonal matrices will have good computational properties. $\square$

There are two types of orthogonal transformations that are widely used in matrix computations: rotators and reflectors. Along with Gaussian elimination operations, these are fundamental building blocks of matrix computations. All matrix computations built upon rotators and/or reflectors, as described in this section, are normwise backward stable. This claim will be discussed later in this section and in Exercises 3.2.70–3.2.73.

We will introduce rotators first because they are simpler.

Rotators

Consider vectors in the plane $\mathbb{R}^2$. The operator that rotates each vector through a fixed angle θ is a linear transformation, so it can be represented by a matrix. Let

$$Q = \begin{bmatrix} q_{11} & q_{12} \\ q_{21} & q_{22} \end{bmatrix}$$

be this matrix. Then Q is completely determined by its action on the two vectors $\begin{bmatrix} 1 \\ 0 \end{bmatrix}$ and $\begin{bmatrix} 0 \\ 1 \end{bmatrix}$, because

$$\begin{bmatrix} q_{11} & q_{12} \\ q_{21} & q_{22} \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} q_{11} \\ q_{21} \end{bmatrix} \quad \text{and} \quad \begin{bmatrix} q_{11} & q_{12} \\ q_{21} & q_{22} \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} q_{12} \\ q_{22} \end{bmatrix}.$$

Since the action of Q is to rotate each vector through the angle θ , clearly (see Figure 3.2)

$$Q \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} \cos \theta \\ \sin \theta \end{bmatrix} \quad \text{and} \quad Q \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} -\sin \theta \\ \cos \theta \end{bmatrix}.$$

Thus $\begin{bmatrix} q_{11} \\ q_{21} \end{bmatrix} = \begin{bmatrix} \cos \theta \\ \sin \theta \end{bmatrix}$ and $\begin{bmatrix} q_{12} \\ q_{22} \end{bmatrix} = \begin{bmatrix} -\sin \theta \\ \cos \theta \end{bmatrix}$; that is,

$$Q = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}.$$

A matrix of this form is called a *rotator* or *rotation*.

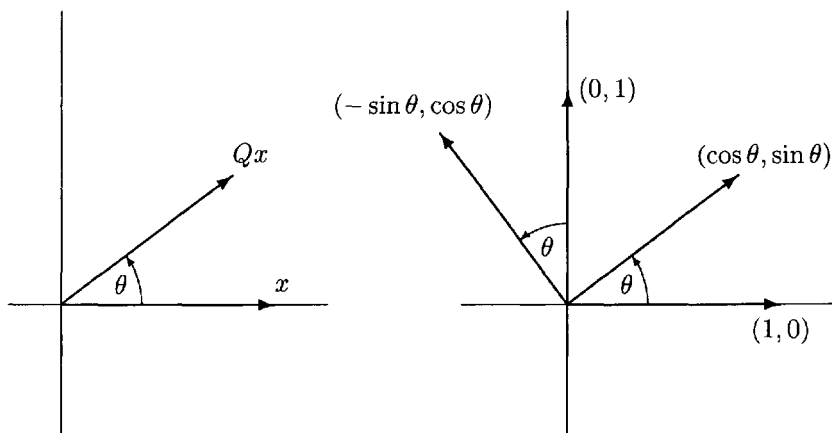


Fig. 3.2 Rotation through angle θ

Exercise 3.2.9 Verify that every rotator is an orthogonal matrix with determinant 1. What does the inverse of a rotator look like? What transformation does it represent? $\square$

Rotators can be used to create zeros in a vector or matrix. For example, if $x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$ is a vector with $x_2 \neq 0$, let us see how to find a rotator

$$Q = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$

such that $Q^T x$ has a zero in its second component: $Q^T x = \begin{bmatrix} y \\ 0 \end{bmatrix}$ for some y . Now

$$Q^T x = \begin{bmatrix} \cos \theta & \sin \theta \\ -\sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} (\cos \theta)x_1 + (\sin \theta)x_2 \\ -(\sin \theta)x_1 + (\cos \theta)x_2 \end{bmatrix},$$

which has the form $\begin{bmatrix} y \\ 0 \end{bmatrix}$ if and only if

$$x_1 \sin \theta = x_2 \cos \theta. \quad (3.2.10)$$

Thus θ can be taken to be $\arctan(x_2/x_1)$ or any other angle satisfying $\tan \theta = x_2/x_1$. But notice that we can determine Q without calculating θ itself; we need only $\cos \theta$ and $\sin \theta$. Clearly the choice $\cos \theta = x_1$ and $\sin \theta = x_2$ would satisfy (3.2.10). However, this choice generally violates the basic trigonometric identity $\cos^2 \theta + \sin^2 \theta = 1$. Therefore we take instead

$$\cos \theta = \frac{x_1}{\sqrt{x_1^2 + x_2^2}} \quad \text{and} \quad \sin \theta = \frac{x_2}{\sqrt{x_1^2 + x_2^2}}, \quad (3.2.11)$$

obtained by dividing our original choice by $\|x\|_2 = \sqrt{x_1^2 + x_2^2}$. Now the basic trigonometric identity is satisfied, and there is a unique $\theta \in [0, 2\pi)$ for which (3.2.11) holds.

Exercise 3.2.12 We have just shown that for every $x \in \mathbb{R}^2$ there exists a rotator Q such that $Q^T x = \begin{bmatrix} y \\ 0 \end{bmatrix}$ for some y .

(a) Give the geometric interpretation of this fact.

(b) Show that if $\cos \theta$ and $\sin \theta$ are given by (3.2.11), then $y = \sqrt{x_1^2 + x_2^2} = \|x\|_2$.

□

Now let us see how we can use a rotator to simplify a matrix

$$A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}.$$

We have seen that there is a rotator Q such that

$$Q^T \begin{bmatrix} a_{11} \\ a_{21} \end{bmatrix} = \begin{bmatrix} r_{11} \\ 0 \end{bmatrix},$$

where $r_{11} = \sqrt{a_{11}^2 + a_{21}^2}$. Define r_{12} and r_{22} by

$$\begin{bmatrix} r_{12} \\ r_{22} \end{bmatrix} = Q^T \begin{bmatrix} a_{12} \\ a_{22} \end{bmatrix},$$

and let

$$R = \begin{bmatrix} r_{11} & r_{12} \\ 0 & r_{22} \end{bmatrix}.$$

Then $Q^T A = R$. This shows that we can transform A to an upper triangular matrix by multiplying it by the orthogonal matrix Q^T . As we shall soon see, it is possible to carry out such a transformation not just for 2-by-2 matrices, but for all $A \in \mathbb{R}^{n \times n}$. That is, for every $A \in \mathbb{R}^{n \times n}$ there is an orthogonal matrix $Q \in \mathbb{R}^{n \times n}$ and an upper triangular matrix $R \in \mathbb{R}^{n \times n}$ such that $Q^T A = R$.

A transformation of this type can be used to solve a system of linear equations $Ax = b$. Multiplying on the left by Q^T , we transform the system to the equivalent system $Q^T Ax = Q^T b$, or $Rx = c$, where $c = Q^T b$. It is a simple matter to calculate c , given b . Then x can be obtained by solving the upper triangular system $Rx = c$ by back substitution.

Another useful viewpoint is reached by rewriting the equations $Q^T A = R$ as

$$A = QR.$$

This shows that A can be expressed as a product QR , where Q is orthogonal and R is upper triangular. The QR decomposition, as it is called, can be used in much

the same way as an LU decomposition to solve a linear system $Ax = b$. If we have a QR decomposition of A , we can rewrite the system as $QRx = b$. Letting $c = Rx$, we can find c by solving $Qc = b$. This doesn't cost much; Q is orthogonal, so $c = Q^T b$. We can then obtain x by solving $Rx = c$ by back substitution. The computations required here are no different from those required by the method outlined in the previous paragraph. Thus we have actually derived a single method from two different points of view.

Example 3.2.13 We will use a QR decomposition to solve the system

$$\begin{bmatrix} 1 & 2 \\ 1 & 3 \end{bmatrix} \begin{bmatrix} z_1 \\ z_2 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}.$$

First we require Q such that $Q^T \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} * \\ 0 \end{bmatrix}$. We use (3.2.11) with $x_1 = 1$ and $x_2 = 1$ to get $c = \cos \theta = 1/\sqrt{2}$ and $s = \sin \theta = 1/\sqrt{2}$. Then

$$Q = \begin{bmatrix} c & -s \\ s & c \end{bmatrix} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix}$$

and

$$R = Q^T A = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 1 & 3 \end{bmatrix} = \frac{1}{\sqrt{2}} \begin{bmatrix} 2 & 5 \\ 0 & 1 \end{bmatrix}.$$

Solving $Qc = b$, we have

$$c = Q^T b = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \frac{1}{\sqrt{2}} \begin{bmatrix} 3 \\ 1 \end{bmatrix}.$$

Finally we solve $Rz = c$ by back substitution and get $z_2 = 1$ and $z_1 = -1$. □

Exercise 3.2.14 Use a QR decomposition to solve the linear system

$$\begin{bmatrix} 2 & 3 \\ 5 & 7 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 12 \\ 29 \end{bmatrix}.$$

□

We now turn our attention to $n \times n$ matrices. A *plane rotator* is a matrix of the form

$$Q = \begin{bmatrix} 1 & & & & & & \\ & \ddots & & & & & \\ & & 1 & & & & \\ & & & c & & -s & \\ & & & & 1 & & \\ & & & & & \ddots & \\ & & & s & & & 1 \\ & & & & & c & \\ & & & & & & 1 \\ & & & & & & & \ddots & \\ & & & & & & & & 1 \end{bmatrix} \begin{matrix} \\ \\ \\ \leftarrow i \\ \\ \leftarrow j \\ \\ \\ \\ \end{matrix}$$

$\uparrow \quad \quad \uparrow$
 $i \quad \quad j$

(3.2.15)

$c = \cos \theta, \quad s = \sin \theta.$

All of the entries that have not been filled in are zeros. Thus a plane rotator looks like an identity matrix, except that one pair of rows and columns contains a rotator. Plane rotators are used extensively in matrix computations. They are sometimes called *Givens rotators* or *Jacobi rotators*, depending on the context³. We will usually drop the adjective “plane” and refer simply to *rotators*.

Exercise 3.2.16 Prove that every plane rotator is orthogonal and has determinant 1. □

Exercise 3.2.17 Let Q be the plane rotator (3.2.15). Show that the transformations $x \rightarrow Qx$ and $x \rightarrow Q^T x$ alter only the i th and j th entries of x and that the effect on these entries is the same as that of the 2-by-2 rotators $\hat{Q} = \begin{bmatrix} c & -s \\ s & c \end{bmatrix}$ and $\hat{Q}^T = \begin{bmatrix} c & s \\ -s & c \end{bmatrix}$ on the vector $\begin{bmatrix} x_i \\ x_j \end{bmatrix}$. □

From Exercise 3.2.17 and (3.2.11) we see that we can transform any vector x to one whose j th entry is zero by applying the plane rotator Q^T , where

$$c = \frac{x_i}{\sqrt{x_i^2 + x_j^2}}, \quad s = \frac{x_j}{\sqrt{x_i^2 + x_j^2}}. \quad (3.2.18)$$

(If $x_i = x_j = 0$, take $c = 1$ and $s = 0$.)

³Rotators applied as described in this section are Givens rotators. Jacobi rotators are discussed in Section 6.6, especially Exercise 6.6.46

It is also important to note the effect of a rotator on a matrix. Let $A \in \mathbb{R}^{n \times m}$, and consider the transformations $A \rightarrow QA$ and $A \rightarrow Q^T A$, where Q is as in (3.2.15). It follows easily from Exercise 3.2.17 that these transformations alter only the i th and j th rows of A . Transposing these results, we see that for $B \in \mathbb{R}^{m \times n}$ the transformations $B \rightarrow BQ$ and $B \rightarrow BQ^T$ alter only the i th and j th columns of B .

Exercise 3.2.19

- Show that the i th and j th rows of QA are linear combinations of the i th and j th rows of A .
- Show that the i th and j th columns of BQ are linear combinations of the i th and j th columns of B .

□

The geometric interpretation of the action of a plane rotator is clear. All vectors lying in the $x_i x_j$ plane are rotated through an angle θ . All vectors orthogonal to the $x_i x_j$ plane are left fixed. A typical vector x is neither in the $x_i x_j$ plane nor orthogonal to it but can be expressed uniquely as a sum $x = p + p^\perp$, where p is in the $x_i x_j$ plane, and $p^\perp$ is orthogonal to it. The plane rotator rotates p through an angle θ and leaves $p^\perp$ fixed.

Theorem 3.2.20 *Let $A \in \mathbb{R}^{n \times n}$. Then there exists an orthogonal matrix Q and an upper triangular matrix R such that $A = QR$.*

Proof. We will sketch a proof in which Q is taken to be a product of rotators. A more detailed proof using reflectors will be given later. Let Q_{21} be a rotator acting on the $x_1 x_2$ plane, such that Q_{21}^T makes the transformation

$$\begin{bmatrix} a_{11} \\ a_{21} \\ a_{31} \\ a_{41} \\ \vdots \\ a_{n1} \end{bmatrix} \rightarrow \begin{bmatrix} * \\ 0 \\ a_{31} \\ a_{41} \\ \vdots \\ a_{n1} \end{bmatrix}.$$

Then $Q_{21}^T A$ has a zero in the $(2, 1)$ position. Similarly we can find a plane rotator Q_{31} , acting in the $x_1 x_3$ plane, such that $Q_{31}^T (Q_{21}^T A)$ has a zero in the $(3, 1)$ position. This rotator does not disturb the zero in the $(2, 1)$ position because Q_{31}^T leaves the second row of $Q_{21}^T A$ unchanged. Continuing in this manner, we create rotators Q_{41} , Q_{51} , $\dots$, Q_{n1} , such that $Q_{n1}^T \cdots Q_{21}^T A$ has zeros in the entire first column, except for the $(1, 1)$ position.

Now we go to work on the second column. Let Q_{32} be a plane rotator acting in the $x_2 x_3$ plane, such that the $(3, 2)$ entry of $Q_{32}^T (Q_{n1}^T \cdots Q_{21}^T A)$ is zero. This rotator does not disturb the zeros that were created previously in the first column. (Why

not?) Continuing on the second column, let $Q_{42}, Q_{52}, \dots, Q_{n2}$ be rotators such that $Q_{n2}^T \cdots Q_{32}^T Q_{n1}^T \cdots Q_{21}^T A$ has zeros in columns 1 and 2 below the main diagonal.

Next we take care of the third column, fourth column, and so on. In all we create rotators $Q_{21}, Q_{31}, \dots, Q_{n1}, Q_{32}, \dots, Q_{n,n-1}$, such that

$$R = Q_{n,n-1}^T Q_{n,n-2}^T \cdots Q_{21}^T A$$

is upper triangular. Let $Q = Q_{21} Q_{31} \cdots Q_{n,n-1}$. Then Q , being a product of orthogonal matrices, is itself orthogonal, and $R = Q^T A$; that is, $A = QR$. $\square$

The proof of Theorem 3.2.20 is constructive; it gives us an algorithm for computing Q and R .

Exercise 3.2.21 Show that the algorithm sketched in the proof of Theorem 3.2.20 takes $O(n^3)$ flops to transform A to R .⁴ $\square$

Exercise 3.2.22 A mathematically precise proof of Theorem 3.2.20 would use induction on n . Sketch an inductive proof of Theorem 3.2.20. $\square$

Reflectors

We begin with the case $n = 2$, just as we did for rotators. Let $\mathcal{L}$ be any line in $\mathbb{R}^2$ that passes through the origin. The operator that reflects each vector in $\mathbb{R}^2$ through the line $\mathcal{L}$ is a linear transformation, so it can be represented by a matrix. Let us determine that matrix. Let v be a nonzero vector lying in $\mathcal{L}$. Then every vector that lies in $\mathcal{L}$ is a multiple of v . Let u be a nonzero vector orthogonal to $\mathcal{L}$. Then $\{u, v\}$ is a basis of $\mathbb{R}^2$, so every $x \in \mathbb{R}^2$ can be expressed as a linear combination of u and v : $x = \alpha u + \beta v$. The reflection of x through $\mathcal{L}$ is $-\alpha u + \beta v$ (see Figure 3.3), so the matrix Q of the reflection must satisfy $Q(\alpha u + \beta v) = -\alpha u + \beta v$ for all α and β . For this it is necessary and sufficient that

$$Qu = -u \quad \text{and} \quad Qv = v.$$

Without loss of generality we can assume that u was chosen to be a unit vector: $\|u\|_2 = 1$.

Consider the matrix $P = uu^T$. P does not have the desired properties, but it brings us closer to our goal. We have $Pu = (uu^T)u = u(u^T u) = u\|u\|_2^2 = u$ and $Pv = (uu^T)v = u(u^T v) = 0$, because u and v are orthogonal. With a little thought we can produce a Q with the desired properties by combining P and the identity matrix. Indeed let $Q = I - 2P$. Then $Qu = u - 2Pu = -u$ and $Qv = v - 2Pv = v$. To summarize, the matrix $Q \in \mathbb{R}^{n \times n}$ that reflects vectors

⁴If you count carefully, you will find that $2n^3 + O(n^2)$ flops are needed. The number of multiplications is about twice the number of additions. However, there are clever ways to implement rotators so that the number of multiplications is halved. Rotators implemented in this special way are called *fast Givens rotations* or simply *fast rotators*. See [33] and other references cited there for a discussion of fast rotators.

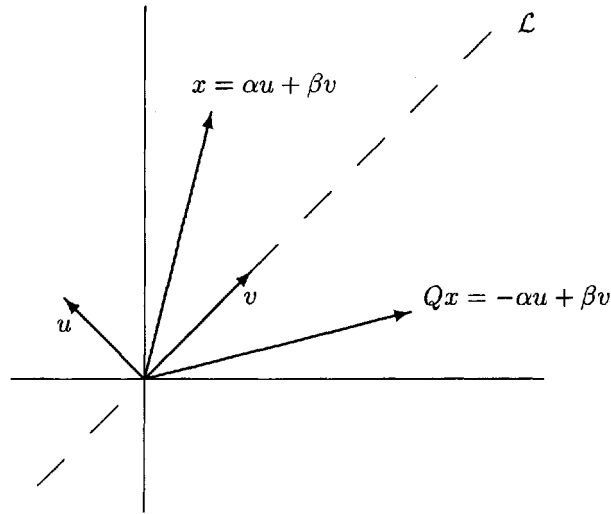


Fig. 3.3 Reflection through a line

through the line $\mathcal{L}$ is given by $Q = I - 2uu^T$, where u is a unit vector orthogonal to $\mathcal{L}$.

Now we are ready to consider n -by- n reflectors.

Theorem 3.2.23 Let $u \in \mathbb{R}^n$ with $\|u\|_2 = 1$, and define $P \in \mathbb{R}^{n \times n}$ by $P = uu^T$. Then

- (a) $Pu = u$.
- (b) $Pv = 0$ if $\langle u, v \rangle = 0$.
- (c) $P^2 = P$.
- (d) $P^T = P$.

Remark 3.2.24 A matrix satisfying $P^2 = P$ is called a *projector* or *idempotent*. A projector that is also symmetric ($P^T = P$) is called an *orthoprojector*. The matrix $P = uu^T$ has rank 1, since its range consists of multiples of u . Thus the properties of P can be summarized by saying that P is a rank-1 orthoprojector. $\square$

Exercise 3.2.25 Prove Theorem 3.2.23. $\square$

Theorem 3.2.26 Let $u \in \mathbb{R}^n$ with $\|u\|_2 = 1$, and define $Q \in \mathbb{R}^{n \times n}$ by $Q = I - 2uu^T$. Then

- (a) $Qu = -u$.

- (b) $Qv = v$ if $\langle u, v \rangle = 0$.
 (c) $Q = Q^T$ (Q is symmetric).
 (d) $Q^T = Q^{-1}$ (Q is orthogonal).
 (e) $Q^{-1} = Q$ (Q is an involution).

Exercise 3.2.27 Prove Theorem 3.2.26. □

Matrices $Q = I - 2uu^T$ ($\|u\|_2 = 1$) are called *reflectors* or *Householder transformations*, after A. S. Householder, who first employed them in matrix computations. Reflectors, like Gauss transforms (Exercise 1.7.49), are rank-one updates of the identity matrix. See Exercise 3.2.64.

The set $\mathcal{H} = \{v \in \mathbb{R}^n \mid \langle u, v \rangle = 0\}$ is an $(n - 1)$ -dimensional subspace of $\mathbb{R}^n$ known as a *hyperplane*. The matrix Q maps each vector x to its reflection through the hyperplane $\mathcal{H}$. This can be visualized by thinking of the case $n = 3$, in which $\mathcal{H}$ is just an ordinary plane through the origin.

In Theorems 3.2.23 and 3.2.26, u was taken to be a unit vector. This is a convenient choice for the statements and proofs of those theorems, but in computations it is usually more convenient not to normalize the vector. The following theorem makes it possible to skip the normalization.

Proposition 3.2.28 Let u be a nonzero vector in $\mathbb{R}^n$, and define $\gamma = 2/\|u\|_2^2$ and $Q = I - \gamma uu^T$. Then Q is a reflector satisfying

- (a) $Qu = -u$.
 (b) $Qv = v$ if $\langle u, v \rangle = 0$.

Proof. Let $\hat{u} = u/\|u\|_2$. Then $\|\hat{u}\| = 1$, and it is a simple matter to check that $Q = I - 2\hat{u}\hat{u}^T$. Properties (a) and (b) follow easily. □

Exercise 3.2.29 Fill in the details of the proof of Proposition 3.2.28. □

Theorem 3.2.30 Let $x, y \in \mathbb{R}^n$ with $x \neq y$ but $\|x\|_2 = \|y\|_2$. Then there is a unique reflector Q such that $Qx = y$.

Proof. We will not prove uniqueness because it is not important to our development (but see Exercise 3.2.63). To establish the existence of Q we must find u such that $(I - \gamma uu^T)x = y$, where $\gamma = 1/\|u\|_2^2$. To see how to proceed, consider the case $n = 2$, which is depicted in part (a) of Figure 3.4. Let $\mathcal{L}$ denote the line that bisects the angle between x and y . The reflection of x through this (and only this) line is y . Thus we require a vector u that is orthogonal to $\mathcal{L}$. It appears that $u = x - y$, or any multiple thereof, is the right choice.

Let $u = x - y$, $\gamma = 2/\|u\|_2^2$, and $Q = I - \gamma uu^T$. To prove that $Qx = y$ we first decompose x into a sum:

$$x = \frac{1}{2}(x - y) + \frac{1}{2}(x + y).$$

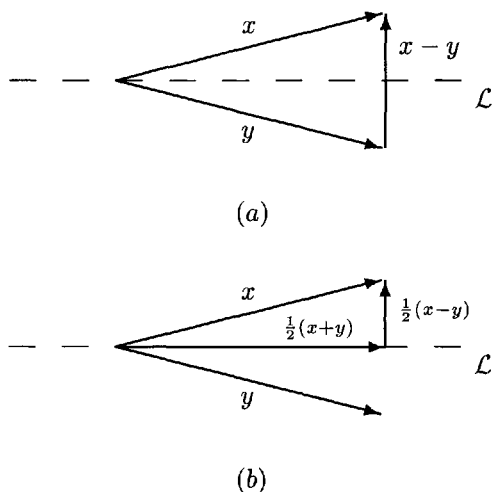


Fig. 3.4 Reflecting x to y .

By part (a) of Proposition 3.2.28, $Q(x - y) = y - x$. Part (b) of Figure 3.4 suggests that $x + y$ is orthogonal to u . To check that this is so, we simply compute the inner product:

$$\begin{aligned}\langle x + y, x - y \rangle &= \langle x, x \rangle + \langle y, x \rangle - \langle x, y \rangle - \langle y, y \rangle \\ &= \|x\|_2^2 + 0 + 0 - \|y\|_2^2 = 0,\end{aligned}$$

because $\|x\|_2 = \|y\|_2$. It follows by part (b) of Proposition 3.2.28 that $Q(x + y) = x + y$. Finally

$$\begin{aligned}Qx &= \frac{1}{2}Q(x - y) + \frac{1}{2}Q(x + y) \\ &= \frac{1}{2}(y - x) + \frac{1}{2}(x + y) = y.\end{aligned}$$

□

It follows easily from this theorem that reflectors can be used to create zeros in vectors and matrices.

Corollary 3.2.31 *Let $x \in \mathbb{R}^n$ be any nonzero vector. Then there exists a reflector Q such that*

$$Q \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} * \\ 0 \\ \vdots \\ 0 \end{bmatrix}.$$

Proof. Let $y = [-\tau, 0, \dots, 0]^T$, where $\tau = \pm\|x\|_2$. By choosing the sign appropriately we can guarantee that $x \neq y$. Clearly $\|x\|_2 = \|y\|_2$. Thus by Theorem 3.2.30, there is a reflector Q such that $Qx = y$. $\square$

Let us take a look at the construction suggested in this proof. The reflector is $Q = I - \gamma uu^T$, where

$$u = x - y = \begin{bmatrix} \tau + x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

and $\gamma = 2/\|u\|_2^2$. Any multiple of this u will generate the same reflector. It turns out to be convenient to normalize u so that its first entry is 1. Thus we will take

$$u = (x - y)/(\tau + x_1) = \begin{bmatrix} 1 \\ x_2/(\tau + x_1) \\ \vdots \\ x_n/(\tau + x_1) \end{bmatrix} \quad (3.2.32)$$

instead. We still have $\gamma = 2/\|u\|_2^2$, but now we are referring to the rescaled u (3.2.32). It is a simple matter to calculate u and γ . We require that $\tau = \pm\|x\|_2$, but we did not specify the sign. In theory either choice can be used (so long as $x \neq y$), but in practice the easiest course is to choose τ so that its sign is the same as that of x_1 . This ensures that cancellation does not occur in the computation of $\tau + x_1$. It also guarantees that when we normalize u by dividing by $\tau + x_1$, we do not divide by zero, nor do we generate dangerously large entries in u .

Exercise 3.2.33 Show that if $\tau = \pm\|x\|_2$ is chosen so that τ and x_1 have the same sign, then all entries of u in (3.2.32) satisfy $|u_i| \leq 1$. $\square$

The calculation of γ is even easier than it looks. Notice that

$$\begin{aligned} \|u\|_2^2 &= \frac{(\tau + x_1)^2 + x_2^2 + \cdots + x_n^2}{(\tau + x_1)^2} \\ &= \frac{\tau^2 + 2\tau x_1 + \|x\|_2^2}{(\tau + x_1)^2}. \end{aligned}$$

But $\tau^2 = \|x\|_2^2$, regardless of which sign was chosen for τ , so $\|u\|_2^2 = 2\tau(\tau + x_1)/(\tau + x_1)^2 = 2\tau/(\tau + x_1)$, and $\gamma = (\tau + x_1)/\tau$.

Before we summarize this computation, let us point out one other potential danger. In order to calculate τ , we need $\|x\|_2 = \sqrt{x_1^2 + \cdots + x_n^2}$. Since squaring doubles the exponent, an overflow can occur if some of the entries are very large. Likewise underflows can occur if some of the entries are very small. Obviously we must avoid overflows; underflows can also occasionally be dangerous.

Example 3.2.34 Consider the calculation of $\|x\|_2$, where

$$x = [10^{-49}, 10^{-50}, 10^{-50}, \dots, 10^{-50}]^T \in \mathbb{R}^{11},$$

on a computer that underflows at 10^{-99} . Any number smaller than that is set to zero. Now $x_1^2 = 10^{-98}$, and $x_2^2 = \dots = x_n^2 = 10^{-100}$, each of which is set to zero by the machine. Thus the computed norm is $\sqrt{x_1^2} = 10^{-49}$. This is in error by almost five percent, as the true value of the norm is $\sqrt{1.1 \times 10^{-98}} \approx 1.0488 \times 10^{-49}$. $\square$

Problems with overflows and underflows in the calculation of $\|x\|_2$ can be avoided by the following simple procedure: Let $\beta = \|x\|_\infty = \max_{1 \leq i \leq n} |x_i|$. If $\beta = 0$, then $\|x\|_2 = 0$. Otherwise let $\hat{x} = (1/\beta)x$. Then $\|x\|_2 = \beta \|\hat{x}\|_2 = \beta \sqrt{\hat{x}_1^2 + \dots + \hat{x}_n^2}$. This scaling procedure eliminates any practical possibility of overflow because $|\hat{x}_i| \leq 1$ for all i . Underflows are still possible, but they occur only when some terms are so much smaller than the largest term ($= 1$) that the error incurred by ignoring them is well below the computer's unit roundoff. Thus these underflows are harmless and can safely be set to zero.

Exercise 3.2.35 Use the scaling procedure outlined above to calculate the norm of the vector x of Example 3.2.34. $\square$

Exercise 3.2.36 The danger of underflows and overflows exists not only in the calculation of reflectors, but in the calculation of rotators as well. Modify the computation (3.2.18) so that the danger of overflows or harmful underflows is eliminated. $\square$

The procedure for calculating Q such that $Qx = y$, where

$$y = \begin{bmatrix} -\tau & 0 & \dots & 0 \end{bmatrix}^T,$$

incorporating the scaling procedure to calculate $\|x\|_2$, is summarized in (3.2.37). To keep the description brief we have used the MATLAB-like notation $x_{i:j}$ to denote entries i through j of vector x .

Given $x \in \mathbb{R}^n$, this algorithm calculates τ , γ , and u such that $Q = (I - \gamma uu^T)$ is a reflector for which $Qx = [-\tau \ 0 \ \dots \ 0]^T$. If $x = 0$, γ is set to zero, giving $Q = I$. Otherwise, γ and u are produced, and u is stored over x . Since u_1 is known to be 1, the operation $x_1 \leftarrow 1$ could be skipped, depending on how the output is to be used.

$$\begin{aligned} & \beta \leftarrow \max_{1 \leq i \leq n} |x_i| \\ & \text{if } (\beta = 0) \\ & \quad \text{then} \\ & \quad \quad \gamma \leftarrow 0 \\ & \quad \text{else} \\ & \quad \quad \begin{cases} x_{1:n} \leftarrow x_{1:n}/\beta \\ \tau \leftarrow \sqrt{x_1^2 + \dots + x_n^2} \\ \text{if } (x_1 < 0) \tau \leftarrow -\tau \\ x_1 \leftarrow \tau + x_1 \\ \gamma \leftarrow x_1/\tau \\ x_{2:n} \leftarrow x_{2:n}/x_1 \\ x_1 \leftarrow 1 \\ \tau \leftarrow \tau\beta \end{cases} \end{aligned} \quad (3.2.37)$$

You can readily see that cancellation cannot occur in this algorithm: The sum $x_1^2 + \cdots + x_n^2$ involves positive numbers only, and we have arranged the computation so that cancellation does not occur in the sum $\tau + x_1$. Therefore the computed τ , γ , and u are accurate.

Exercise 3.2.38 Check that the total work to execute Algorithm (3.2.37) is $O(n)$. □

Algorithm (3.2.37) does not calculate Q explicitly, but then Q is seldom needed; it suffices to save γ and u . For example, when reflectors are used to compute the QR decomposition of a matrix, we need to multiply each reflector Q by a matrix B that is a submatrix of the matrix that is undergoing a transformation to upper-triangular form. Let us see how we can compute QB economically. Suppose $Q \in \mathbb{R}^{n \times n}$, and $B \in \mathbb{R}^{n \times m}$. We have $QB = (I - \gamma uu^T)B = B - \gamma uu^T B$, so the main task is to calculate the product $\gamma uu^T B$. There are good and bad ways to do this. The first good thing we can do is absorb the scalar γ into one of the vectors. Let $v^T = \gamma u^T$, so that $QB = B - uv^T B$.

Exercise 3.2.39 The amount of work required to compute $uv^T B$ depends dramatically upon the order in which the operations are performed. Suppose, as above, that $u \in \mathbb{R}^n$, $v \in \mathbb{R}^n$, and $B \in \mathbb{R}^{n \times m}$.

- Show that if we compute $(uv^T)B$, the intermediate result is an $n \times n$ matrix, and the total computation requires about $2n^2 m$ flops.
- Show that if we compute $u(v^T B)$, the intermediate result is a $1 \times m$ matrix, and the total computation requires about $3nm$ flops. This arrangement clearly requires much less storage space and arithmetic than the arrangement of part (a).
- Show that the computation $QB = B - uv^T B$ requires about $4nm$ flops in all if it is done in the economical way.
- How many flops are needed to compute QB if Q is stored in the conventional way as an $n \times n$ matrix?

□

From this exercise we see that the economical approach to computing QB is to compute $B - u(v^T B)$. The operations are summarized in the following algorithm.

Algorithm to calculate QB and store it over B , where $B \in \mathbb{R}^{n \times m}$ and $Q = (I - \gamma uu^T)$. An auxiliary vector $v \in \mathbb{R}^n$ is used.

$$\begin{aligned} v^T &\leftarrow \gamma u^T \\ v^T &\leftarrow v^T B \\ B &\leftarrow B - uv^T \end{aligned} \tag{3.2.40}$$

The total flop count is about $4nm$, which is dramatically less expensive than assembling Q explicitly and multiplying Q by B in the conventional manner. This saving is possible because the reflector Q is a rank-one update of the identity (Exercise 3.2.64).

Algorithm (3.2.40) can be implemented as either column-oriented or row-oriented code, depending upon the loop order in the two matrix-matrix multiplications. If column-oriented code is chosen, the auxiliary vector v can be replaced by a single auxiliary scalar, and the flop count can be lowered marginally, by careful organization. However, these modifications will not necessarily result in faster code.

Exercise 3.2.41

- (a) Find a reflector Q that maps the vector $[3, 4, 1, 3, 1]^T$ to a vector of the form $[-\tau, 0, 0, 0, 0]^T$. You need not rescale x . Write Q two ways: (i) in the form $I - \gamma uu^T$ and (ii) as a completely assembled matrix.
- (b) Let $a = [0, 2, 1, -1, 0]^T$. Calculate Qa two different ways: (i) the efficient way, using $I - \gamma uu^T$, and (ii) using the assembled matrix Q .

□

QR Decomposition by Reflectors

Theorem 3.2.20 states that every $A \in \mathbb{R}^{n \times n}$ can be expressed as a product $A = QR$, where Q is orthogonal and R is upper triangular. We used rotators to prove Theorem 3.2.20, but we promised a second proof based on reflectors. We are now ready for that proof.

Proof of Theorem 3.2.20 using reflectors. The proof is by induction on n . When $n = 1$, take $Q = [1]$ and $R = [a_{11}]$ to get $A = QR$. Now we will take an arbitrary $n \geq 2$ and show that the theorem holds for n -by- n matrices if it holds for $(n - 1)$ -by- $(n - 1)$ matrices. Let $Q_1 \in \mathbb{R}^{n \times n}$ be a reflector that creates zeros in the first column of A :

$$Q_1 \begin{bmatrix} a_{11} \\ a_{21} \\ \vdots \\ a_{n1} \end{bmatrix} = \begin{bmatrix} -\tau_1 \\ 0 \\ \vdots \\ 0 \end{bmatrix}. \quad (3.2.42)$$

Recalling that Q_1 is symmetric, we see that $Q_1^T A$ has the form

$$Q_1^T A = Q_1 A = \left[\begin{array}{c|ccc} -\tau_1 & \hat{a}_{12} & \cdots & \hat{a}_{1n} \\ \hline 0 & & & \\ \vdots & & \hat{A}_2 & \\ 0 & & & \end{array} \right]. \quad (3.2.43)$$

By the induction hypothesis $\hat{A}_2$ has a QR decomposition: $\hat{A}_2 = \hat{Q}_2 \hat{R}_2$, where $\hat{Q}_2$ is orthogonal and $\hat{R}_2$ is upper triangular. Define $\tilde{Q}_2 \in \mathbb{R}^{n \times n}$ by

$$\tilde{Q}_2 = \left[\begin{array}{c|ccc} 1 & 0 & \cdots & 0 \\ \hline 0 & & & \\ \vdots & & \hat{Q}_2 & \\ 0 & & & \end{array} \right].$$

Then obviously $\tilde{Q}_2$ is orthogonal, and

$$\begin{aligned}\tilde{Q}_2^T Q_1^T A &= \left[\begin{array}{c|ccc} 1 & 0 & \cdots & 0 \\ \hline 0 & & & \\ \vdots & \hat{Q}_2 & & \\ 0 & & & \end{array} \right] \left[\begin{array}{c|ccc} -\tau_1 & \hat{a}_{12} & \cdots & \hat{a}_{1n} \\ \hline 0 & & & \\ \vdots & & \hat{A}_2 & \\ 0 & & & \end{array} \right] \\ &= \left[\begin{array}{c|ccc} -\tau_1 & \hat{a}_{12} & \cdots & \hat{a}_{1n} \\ \hline 0 & & & \\ \vdots & & \hat{R}_2 & \\ 0 & & & \end{array} \right].\end{aligned}\quad (3.2.44)$$

This matrix is upper triangular; let us call it R . Let $Q = Q_1 \tilde{Q}_2$. Then Q is orthogonal, and $Q^T A = R$. Therefore $A = QR$. $\square$

This proof can be turned into an algorithm for constructing Q and R . Step 1 calculates a reflector $Q_1 = I - \gamma_1 u^{(1)} u^{(1)T}$ to perform the task (3.2.42) and then uses Q_1 to make the transformation (3.2.43). In practice we need not form Q_1 explicitly; we can just save $-\tau_1$, γ_1 , and $u^{(1)}$, which were calculated by (3.2.37) with x taken to be the first column of A . There is no need to transform the first column to the form $[-\tau_1 \ 0 \ \cdots \ 0]^T$, as shown in (3.2.43), at least the zeros need not be stored. We can store $-\tau_1 = r_{11}$ in the $(1, 1)$ position, and the rest of the first column can be used to store $u^{(1)}$. Since $u_1^{(1)} = 1$, there is no need to store it, and there is just enough room for the rest of $u^{(1)}$ in the rest of the first column of A . The transformation is achieved simply by dividing each of $a_{21}, \dots, a_{n1}$ by $\tau_1 + a_{11}$. The rest of A can be transformed as indicated in (3.2.43). This is done by applying reflector Q_1 to the submatrix of A consisting of columns 2 through n , using algorithm (3.2.40). This is the expensive part of the computation. Since $m = n - 1$ columns are involved here, the total flop count for step 1 is about $4n^2$. Since the cost of building the reflector $O(n)$, we ignore it.

The rest of the algorithm consists of reducing $\hat{A}_2$ to upper triangular form. Comparing (3.2.43) with (3.2.44), we see that the first row and column remain unchanged after step 1, so they can be ignored. Step 2 is identical to step 1, except that it operates on $\hat{A}_2$ to convert it to the form

$$\left[\begin{array}{c|ccc} -\tau_2 & \hat{a}_{23} & \cdots & \hat{a}_{2n} \\ \hline 0 & & & \\ \vdots & & \hat{A}_3 & \\ 0 & & & \end{array} \right].$$

In practice we use the zero part of the first column to store $u^{(2)}$. The cost of step 2 is about $4(n-1)^2$ flops, since it is identical to step 1, except that it acts on an $(n-1) \times (n-1)$ matrix. Step 3 is identical to step 2, except that it acts on the smaller matrix $\hat{A}_3$. The cost of the third step is about $4(n-2)^2$ flops.

After $n - 1$ steps the matrix has been transformed to upper triangular form. The array that originally held A now holds R and the vectors $u^{(1)}, \dots, u^{(n-1)}$, stripped of their leading 1's. Another array holds $\gamma_1, \dots, \gamma_{n-1}$. Clearly $R = Q_{n-1}Q_{n-2} \cdots Q_1 A$, where $Q_1 = I - \gamma_1 u^{(1)} u^{(1)T}$,

$$Q_2 = \left[\begin{array}{c|ccc} 1 & 0 & \cdots & 0 \\ \hline 0 & & & \\ \vdots & & I - \gamma_2 u^{(2)} u^{(2)T} & \\ 0 & & & \end{array} \right],$$

and in general

$$Q_i = \left[\begin{array}{c|ccc} I_{i-1} & & & 0 \\ \hline 0 & & I - \gamma_i u^{(i)} u^{(i)T} & \end{array} \right],$$

where I_{i-1} denotes the identity matrix of dimension $i - 1$. Letting

$$Q = Q_1 Q_2 \cdots Q_{n-1},$$

we have $Q^T = Q_{n-1}^T \cdots Q_1^T = Q_{n-1} \cdots Q_1$, $R = Q^T A$, and finally $A = QR$. As we shall see, there is no need to assemble Q explicitly.

The algorithm can be summarized as follows.

Algorithm to calculate the QR decomposition of $A \in \mathbb{R}^{n \times n}$ using reflectors

$$\begin{aligned} &\text{for } k = 1, \dots, n-1 \\ &\quad \left[\begin{array}{l} \text{Determine a reflector } Q_k = I - \gamma_k u^{(k)} u^{(k)T} \text{ such that} \\ Q_k [a_{kk} \cdots a_{nk}]^T = [-\tau_k \ 0 \cdots 0]^T \quad (\text{see (3.2.37)}) \\ \text{Store } u^{(k)} \text{ over } a_{k:n,k}, \text{ as in (3.2.37).} \\ a_{k:n,k+1:n} \leftarrow Q_k a_{k:n,k+1:n} \quad (\text{see (3.2.40)}) \\ a_{kk} \leftarrow -\tau_k \end{array} \right. \quad (3.2.45) \\ &\quad \gamma_n \leftarrow a_{nn} \end{aligned}$$

We have used the MATLAB-like notation $a_{i:j,k:m}$ to denote the submatrix consisting of the intersection of rows i through j with columns k through m . γ_n functions as a flag. If γ_n or any other γ_i is zero, A is singular. (Why?)

Recalling that the flop count for step one is $4n^2$, for step two is $4(n-1)^2$, and so on, we see that the total flop count for (3.2.45) is about $4n^2 + 4(n-1)^2 + 4(n-2)^2 + \cdots \approx 4n^3/3$, twice that of an LU decomposition.

Most of the work is in the operations $a_{k:n,k+1:n} \leftarrow Q_k a_{k:n,k+1:n}$, which should be organized as indicated in (3.2.40). These operations can be done in a column-oriented or row-oriented way.

There are also block variants of the algorithm that are suitable for application to large matrices. These delay the application of the reflectors. Once a certain number of reflectors have been constructed, the whole batch is applied at once in a way that is rich in matrix-matrix multiplications. These operations can be organized into blocks for efficient use of high-speed cache memory and parallel processors. The main idea is worked out in Exercises 3.2.67 and 3.2.68. The LAPACK [1] routine DGEQRF is a Fortran implementation of the QR decomposition by blocks.

Let us see how the QR decomposition, as calculated using reflectors, can be used to solve the system of linear equations $Ax = b$. This is admittedly an expensive way to solve a linear system, but it is a good illustration because the operations are similar to those that we will use to solve least squares problems for overdetermined systems in Section 3.3. Since $A = QR$, our task is to solve $QRx = b$. Letting $c = Rx$, we must first solve $Qc = b$ for c . Then we can solve $Rx = c$ for x . First of all $c = Q^T b = Q_{n-1} \cdots Q_2 Q_1 b$, so we can apply the reflectors $Q_1, Q_2, \dots, Q_{n-1}$ to b successively to obtain c . Naturally we use (3.2.40) with $m = 1$. Finally we solve $Rx = c$ for x by back substitution.

Applying Q_1 costs $4n$ flops. Applying Q_2 costs only $4(n-1)$ flops because it is really only an $(n-1) \times (n-1)$ reflector; it does not touch the first entry of $Q_1 b$. Similarly Q_3 does not touch the first two entries of $Q_2 Q_1 b$, and the cost of applying it is $4(n-2)$ flops. The total cost of calculating c is therefore $4n + 4(n-1) + 4(n-2) + \cdots \approx 2n^2$. This is the same as it would have cost to calculate $c = Q^T b$ if Q had been available explicitly. The cost of solving for x by back substitution is about n^2 flops. Thus these steps are inexpensive, compared with the cost of computing the QR decomposition.

Uniqueness of the QR Decomposition

For nonsingular matrices the QR decomposition theorem can be strengthened to include the uniqueness of Q and R . This fact is of considerable theoretical importance.

Theorem 3.2.46 *Let $A \in \mathbb{R}^{n \times n}$ be nonsingular. Then there exist unique $Q, R \in \mathbb{R}^{n \times n}$ such that Q is orthogonal, R is upper triangular with positive main-diagonal entries, and $A = QR$.*

Proof. By Theorem 3.2.20, $A = \hat{Q}\hat{R}$, where $\hat{Q}$ is orthogonal and $\hat{R}$ is upper triangular, but $\hat{R}$ does not necessarily have positive main-diagonal entries. Since A is nonsingular, $\hat{R}$ must also be nonsingular, so its main-diagonal entries are nonzero, at least. Let D be the diagonal matrix given by

$$d_{ii} = \begin{cases} 1 & \text{if } \hat{r}_{ii} > 0, \\ -1 & \text{if } \hat{r}_{ii} < 0. \end{cases}$$

Then $D = D^T = D^{-1}$ is orthogonal. Let $Q = \hat{Q}D^{-1}$ and $R = D\hat{R}$. Then Q is orthogonal, R is upper triangular with $r_{ii} = d_{ii}\hat{r}_{ii} > 0$, and $A = QR$. This establishes existence.

Uniqueness can be proved using the same ideas (see Exercise 3.2.47). For variety we will take a different approach, which exploits an interesting connection between the QR and Cholesky decompositions. Suppose $A = Q_1 R_1 = Q_2 R_2$, where Q_1 and Q_2 are orthogonal, and R_1 and R_2 are upper triangular with positive main-diagonal entries. $A^T A$ is a positive definite matrix, and $A^T A = R_1^T R_1$, since $Q_1^T Q_1 = I$. Thus R_1 is the Cholesky factor of $A^T A$. The same arguments can be applied to R_2 , so R_2 is also the Cholesky factor of A . By uniqueness of the Cholesky decomposition, $R_1 = R_2$. Finally $Q_1 = AR_1^{-1} = AR_2^{-1} = Q_2$. $\square$

Exercise 3.2.47 This exercise leads to another uniqueness proof for Theorem 3.2.46.

- (a) Suppose $B \in \mathbb{R}^{n \times n}$ is both orthogonal and upper triangular. Prove that B must be a diagonal matrix whose main-diagonal entries are ± 1 .
- (b) Suppose $Q_1 R_1 = Q_2 R_2$, where Q_1 and Q_2 are both orthogonal, and R_1 and R_2 are both upper triangular and nonsingular. Show that there is a diagonal matrix D with main diagonal entries ± 1 such that $R_2 = D R_1$ and $Q_1 = Q_2 D$.

□

Exercise 3.2.48

- (a) Let $A = \begin{bmatrix} 1 & 2 \\ 1 & 3 \end{bmatrix}$. Find a reflector $\hat{Q}$ and an upper triangular $\hat{R}$ such that $A = \hat{Q}\hat{R}$. Assemble $\hat{Q}$ and simplify it.

$$\left(\text{Solution: } \hat{Q} = \frac{1}{\sqrt{2}} \begin{bmatrix} -1 & -1 \\ -1 & 1 \end{bmatrix}, \quad \hat{R} = \frac{1}{\sqrt{2}} \begin{bmatrix} -2 & -5 \\ 0 & 1 \end{bmatrix} \right)$$

- (b) Compare your solution from part (a) with the QR decomposition of the same matrix (obtained using a rotator) in Example 3.2.13. Find a diagonal matrix D with main diagonal entries ± 1 such that $Q = \hat{Q}D$ and $\hat{R} = DR$.

□

Exercise 3.2.49 The MATLAB command `qr` performs the QR decomposition of a matrix. In the interest of keeping the output uncomplicated, MATLAB delivers the assembled Q matrix. Try out the following MATLAB commands, for example.

```
n = 7
A = randn(n)
[Q,R] = qr(A)
Q'*Q
norm(eye(n)-Q'*Q)
norm(A-Q*R)
```

Notice that MATLAB does not bother to make the main-diagonal entries of R positive, since this is not essential for most applications. A number of other ways to use the `qr` command will be discussed in the coming section. For further information type `help qr`. □

Stability of Computations with Rotators and Reflectors

The numerical properties of both rotators and reflectors are excellent. A detailed analysis was carried out by Wilkinson [81, pp. 126–162]. See also [41]. We will content ourselves with a brief summary. Some details are worked out in Exercises 3.2.70–3.2.73.

Suppose we wish to convert A to QA , where Q is either a rotator determined by (3.2.18) or a reflector determined by (3.2.37). In the case of the reflector, (3.2.40) should be used to calculate QA . In both cases Wilkinson has shown that

$$\text{fl}(QA) = Q(A + E),$$

where $\|E\|_2/\|A\|_2$ is tiny. In other words, the computed result is the exact result of applying Q to a matrix that is a very slight perturbation of A . That is, the computation of QA is normwise backward stable.]

The stability is preserved under repeated applications of rotators and/or reflectors. Consider, for example, the computation of Q_2Q_1A . First of all, $\text{fl}(Q_1A) = Q_1(A + E_1)$. Thus $\text{fl}(Q_2Q_1A) = \text{fl}(Q_2(Q_1(A + E_1))) = Q_2(Q_1(A + E_1) + E_2) = Q_2Q_1(A + E_1 + Q_1^TE_2) = Q_2Q_1(A + E)$, where $E = E_1 + Q_1^TE_2$. Since $\|E\|_2 \leq \|E_1\|_2 + \|E_2\|_2$, we can assert that $\|E\|_2/\|A\|_2$ is tiny.

Many of the algorithms in this book consist of repeated modifications of a matrix by reflectors or rotators. The two QR decomposition algorithms discussed in this section are of this type, and so are many of the eigenvalue algorithms that are discussed in subsequent chapters. The preceding remarks imply that all algorithms of this type are normwise backward stable.

The Complex Case

The following exercises develop the complex analogues of the results obtained in this section. The *inner product* on $\mathbb{C}^n$ is defined by

$$\langle x, y \rangle = \sum_{i=1}^n x_i \bar{y}_i, \quad \{x = [x_1 \cdots x_n]^T, y = [y_1 \cdots y_n]^T\}$$

where the bar denotes complex conjugation. Clearly $\langle x, y \rangle = y^*x = \overline{x^*y}$, where y^* denotes the conjugate transpose of y .

Exercise 3.2.50 Show that the inner product on $\mathbb{C}^n$ satisfies the following properties.

- (a) $\langle x, y \rangle = \overline{\langle y, x \rangle}$.
- (b) $\langle \alpha_1 x_1 + \alpha_2 x_2, y \rangle = \alpha_1 \langle x_1, y \rangle + \alpha_2 \langle x_2, y \rangle$.
- (c) $\langle x, \alpha_1 y_1 + \alpha_2 y_2 \rangle = \bar{\alpha}_1 \langle x, y_1 \rangle + \bar{\alpha}_2 \langle x, y_2 \rangle$.
- (d) $\langle x, x \rangle$ is real, $\langle x, x \rangle \geq 0$, and $\langle x, x \rangle = 0$ if and only if $x = 0$.
- (e) $\sqrt{\langle x, x \rangle} = \|x\|_2$.

□

The complex analogues of orthogonal matrices are unitary matrices. A matrix $U \in \mathbb{C}^{n \times n}$ is *unitary* if $UU^* = I$, where U^* is the conjugate transpose of U .

Equivalent statements are $U^*U = I$ and $U^* = U^{-1}$. Notice that the orthogonal matrices are just the real unitary matrices.

Exercise 3.2.51

- (a) Show that if U is unitary, then U^{-1} is unitary.
- (b) Show that if U_1 and U_2 are unitary, then U_1U_2 is unitary.

□

Exercise 3.2.52 Show that if U is unitary, then $|\det(U)| = 1$.

□

Exercise 3.2.53 Show that if $U \in \mathbb{C}^{n \times n}$ is unitary and $x, y \in \mathbb{C}^n$, then

- (a) $\langle Ux, Uy \rangle = \langle x, y \rangle$.
- (b) $\|Ux\|_2 = \|x\|_2$.
- (c) $\|U\|_2 = \|U^{-1}\|_2 = \kappa_2(U) = 1$.

Thus unitary matrices preserve the 2-norm and the inner product.

□

Exercise 3.2.54 (Complex Rotators) Given a nonzero $\begin{bmatrix} a \\ b \end{bmatrix} \in \mathbb{C}^2$ define $U \in \mathbb{C}^{2 \times 2}$ by

$$U = \frac{1}{\gamma} \begin{bmatrix} a & -\bar{b} \\ b & \bar{a} \end{bmatrix}, \quad \text{where } \gamma = \sqrt{|a|^2 + |b|^2}.$$

Verify that

- (a) U is unitary.
- (b) $\det(U) = 1$.
- (c) $U^* \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} \gamma \\ 0 \end{bmatrix}$.

□

The extension to complex plane rotators in $\mathbb{C}^{n \times n}$ is obvious.

Exercise 3.2.55 (Complex Reflectors) Let $u \in \mathbb{C}^n$ with $\|u\|_2 = 1$, and define $Q \in \mathbb{C}^{n \times n}$ by $Q = I - 2uu^*$. Verify that

- (a) $Qu = -u$.
- (b) $Qv = v$ if $\langle u, v \rangle = 0$.
- (c) $Q = Q^*$ (Q is Hermitian).
- (d) $Q^* = Q^{-1}$ (Q is unitary).

(e) $Q^{-1} = Q$ (Q is an involution).

□

Exercise 3.2.56

- (a) Prove that if $x, y \in \mathbb{C}^n$, $x \neq y$, $\|x\|_2 = \|y\|_2$, and $\langle x, y \rangle$ is real, then there exists a complex reflector Q such that $Qx = y$.
- (b) Let $x \in \mathbb{C}^n$ be a nonzero vector. Express x_1 in polar form as $x_1 = re^{i\theta}$ (with $\theta = 0$ if $x_1 = 0$). Let $\tau = \|x\|_2 e^{i\theta}$ and $y = [-\tau \ 0 \ \cdots \ 0]^T$. Show that $\|x\| = \|y\|$, $x \neq y$, and $\langle x, y \rangle$ is real. Write an algorithm that determines a reflector Q such that $Qx = y$. This is the complex analogue of (3.2.37).

□

Exercise 3.2.57 Show that for every $A \in \mathbb{C}^{n \times n}$ there exist unitary Q and upper triangular R such that $A = QR$. Write a constructive proof using either rotators or reflectors. □

If A is nonsingular, then R can be chosen so that its main-diagonal entries are (real and) positive, in which case Q and R are unique. This is the complex analogue of Theorem 3.2.46.

Theorem 3.2.58 *Let $A \in \mathbb{C}^{n \times n}$ be nonsingular. Then there exist unique $Q, R \in \mathbb{C}^{n \times n}$ such that Q is unitary, R is upper triangular with real, positive main-diagonal entries, and $A = QR$.*

Exercise 3.2.59 Prove Theorem 3.2.58. □

Exercise 3.2.60 Let $A \in \mathbb{C}^{n \times n}$ be nonsingular. Suppose $A = QR = \tilde{Q}\tilde{R}$, where Q and $\tilde{Q}$ are unitary, and R and $\tilde{R}$ are upper triangular.

1. Show that $Q = \tilde{Q}D$, where D is a unitary, diagonal matrix.
2. Show that the j th column of $\tilde{Q}$ is a multiple of the j th column of Q for $j = 1, \dots, n$.

□

Exercise 3.2.61 MATLAB will happily compute QR decompositions of complex matrices. Try the following commands, for example.

```
n = 4
A = randn(n) + i*randn(n)
[Q,R] = qr(A)
Q'
Q'*Q
norm(eye(n)-Q'*Q)
norm(A-Q*R)
```

MATLAB does not bother to make the main-diagonal entries of R positive or even real. Notice that Q' is the *conjugate* transpose of Q . □

Exercise 3.2.62 Let $u \in \mathbb{C}^n$ with $\|u\|_2 = 1$. Then $I - \gamma uu^*$ is unitary if $\gamma = 2$ or 0 . Find the set of all $\gamma \in \mathbb{C}$ for which $I - \gamma uu^*$ is unitary. $\square$

Additional Exercises

Exercise 3.2.63 Show that if $Qx = y$, where $Q = I - \gamma uu^T \in \mathbb{R}^{n \times n}$, then u must be a multiple of $x - y$. Since all multiples of $x - y$ generate the same reflector, this establishes the uniqueness part of Theorem 3.2.30. $\square$

Exercise 3.2.64 Some of the most important transforming matrices are rank-one updates of the identity matrix.

- Recall that the rank of a matrix is equal to the number of linearly independent columns. Prove that $A \in \mathbb{R}^{n \times n}$ has rank one if and only if there exist nonzero vectors $u, v \in \mathbb{R}^n$ such that $A = uv^T$. To what extent is there flexibility in the choice of u and v ?
- A matrix of the form $G = I - uv^T$ is called a *rank-one update of the identity* for obvious reasons. Prove that if G is singular, then $Gx = 0$ if and only if x is a multiple of u . Prove that G is singular if and only if $v^T u = 1$.
- Show that if $G = I - uv^T$ is nonsingular, then G^{-1} has the form $I - \beta uv^T$. Give a formula for β .
- Prove that if G is orthogonal, then u and v are proportional, i.e. $v = \rho u$ for some nonzero scalar ρ . Show that if $\|u\|_2 = 1$, then $\rho = 2$.
- Show that if $G = I - uv^T$ and $W \in \mathbb{R}^{n \times m}$, then GW can be computed in about $4nm$ flops if the arithmetic is done in an efficient way.
- Consider the task of introducing zeros into a vector: Given $x \in \mathbb{R}^n$, we want to find a $G = I - uv^T$ such that

$$G \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} \beta \\ 0 \\ \vdots \\ 0 \end{bmatrix} \quad (3.2.65)$$

for some nonzero β . Show that G performs the transformation (3.2.65) if and only if

$$u = (1/\gamma)(x - \beta e_1), \quad \text{where } \gamma = v^T x \neq 0, \quad (3.2.66)$$

and $e_1 = [1, 0, \dots, 0]^T$.

- In (3.2.65) β can be chosen freely. In this section we have already seen how to choose β, u , and v to build a reflector to do this task. Now consider another possibility, assuming $x_1 \neq 0$. Take $\beta = x_1$ and $v = e_1$. What form must u

now have, if (3.2.66) is to be satisfied? Show that the resulting G is a Gauss transform (cf. Exercise 1.7.47).

□

Exercise 3.2.67 . This exercise shows how to accumulate reflectors so that they can be applied in a batch using operations that are rich in matrix-matrix multiplications.

- (a) Let $G = I + WY^T$, where $W, Y \in \mathbb{R}^{n \times j}$. Show that $G = I + w_1 y_1^T + w_2 y_2^T + \cdots + w_j y_j^T$, where $w_1, \dots, w_j$ and $y_1, \dots, y_j$ are column vectors. Thus G is a *rank j update* of the identity.
- (b) Let G be as in part (a), with $1 \ll j \ll n$, and let $B \in \mathbb{R}^{n \times m}$. Show that the product GB can be calculated in about $4njm$ flops if the operations are organized carefully. What size matrix-matrix products and sums are performed in the process? (The bigger they are, the more efficiently we can use the computer's cache.) How many flops would it take to compute GB if G were stored as an $n \times n$ matrix in the standard way?
- (c) Show that if $Q = I - \gamma uu^T$ and $G = I + WY^T$, then $QG = I + WY^T + uv^T$, where $v = -\gamma(u + YW^T u)$. How many flops does it take to compute v ? Show that $QG = I + \tilde{W}\tilde{Y}^T$, where $\tilde{W}, \tilde{Y} \in \mathbb{R}^{n \times (j+1)}$.
- (d) Prove that any product of j elementary reflectors can be expressed in the form $I + WY^T$, where $W, Y \in \mathbb{R}^{n \times j}$. Use induction on j .
- (e) Let G be a product of j elementary reflectors. How many flops are needed to compute GB , where $B \in \mathbb{R}^{n \times m}$, if G is applied as a series of j reflectors, one after the other? How does this compare with the cost of applying G in the form $I + WY^T$? How many additional flops are needed to construct $I + WY^T$ via the operations indicated in part (c)? (This shows that the extra work is insignificant if $j \ll n$ but can become significant if j is made too large.)
- (f) State briefly how (3.2.45) would have to be modified so that the reflectors are applied in batches of j .
- (g) Give a simple expression for G^{-1} , if $G = I + WY^T$ is orthogonal.

□

Exercise 3.2.68 The LAPACK [1] routines use a slightly different representation than the one developed in the previous exercise. Suppose $Q = Q_j Q_{j-1} \cdots Q_1$ is a product of reflectors $Q_i = I - \gamma u_i u_i^T$. Proceeding more or less as in the previous exercise, prove by induction on j that $Q = I - ULU^T$, where $U = [u_1 \cdots u_j] \in \mathbb{R}^{n \times j}$, and $L \in \mathbb{R}^{j \times j}$ is lower triangular. How can Q^{-1} be represented? □

Exercise 3.2.69 Show that every 2-by-2 reflector has the form $\begin{bmatrix} c & s \\ s & -c \end{bmatrix}$, where $c^2 + s^2 = 1$. How does the reflector $Q = \begin{bmatrix} c & s \\ s & -c \end{bmatrix}$ act on the unit vectors $\begin{bmatrix} 1 \\ 0 \end{bmatrix}$ and $\begin{bmatrix} 0 \\ 1 \end{bmatrix}$?

Contrast the action of Q with that of the rotator $\tilde{Q} = \begin{bmatrix} c & -s \\ s & c \end{bmatrix}$. Draw a picture. By examining your picture, determine the line through which the vectors are reflected by Q . Assuming $c > 0$ and $s > 0$, show that this line has slope $\tan(\theta/2)$, where $\theta = \arccos c = \arcsin s$. $\square$

Exercise 3.2.70 This and subsequent exercises demonstrate the backward stability of multiplication by rotators, reflectors, and other orthogonal transformations. We begin with matrix multiplication in general. Let $V \in \mathbb{R}^{n \times n}$ and $x \in \mathbb{R}^n$.

- (a) Suppose we calculate $\text{fl}(Vx)$ on an ideal computer (Section 2.5) with unit roundoff u . Using (2.5.3) and Proposition 2.7.1, show that $\text{fl}(Vx) = Vx + b$, where b is tiny; specifically, the i th component of b satisfies

$$b_i = \sum_{j=1}^n v_{ij} x_j \gamma_{ij},$$

where $|\gamma_{ij}| \leq nu + O(u^2)$.

- (b) Continuing from part (a), show that $|b| \leq \gamma |V| |x|$, where $\gamma = \max |\gamma_{ij}|$, and $\|b\|_1 \leq \|V\|_1 \|x\|_1 (nu + O(u^2))$.
- (c) Show that $\text{fl}(Vx) = V(x + \delta x)$, where $\delta x (= V^{-1}b)$ satisfies $\|\delta x\|_1 / \|x\|_1 \leq \kappa_1(V)(nu + O(u^2))$.
- (d) Let $A \in \mathbb{R}^{n \times n}$. Using the result of part (c), show that $\text{fl}(VA) = V(A + E)$, where $\|E\|_1 / \|A\|_1 \leq \kappa_1(V)(nu + O(u^2))$.

We conclude that multiplication by V is backward stable if V is well conditioned. Orthogonal matrices are the ultimate well-conditioned matrices. $\square$

Exercise 3.2.71 Use inequalities from Exercise 2.1.32 to prove parts (a) and (b). Then use these inequalities along with the result of Exercise 3.2.70 to work part (c).

- (a) For all $E \in \mathbb{R}^{n \times n}$ and nonzero $A \in \mathbb{R}^{n \times n}$, $\|E\|_2 / \|A\|_2 \leq n \|E\|_1 / \|A\|_1$.
- (b) For all nonsingular $V \in \mathbb{R}^{n \times n}$, $\kappa_1(V) \leq n \kappa_2(V)$.
- (c) Show that if $Q \in \mathbb{R}^{n \times n}$ is orthogonal, then $\text{fl}(QA) = Q(A + E)$, where

$$\|E\|_2 / \|A\|_2 \leq n^3 u + O(u^2).$$

Of course, the factor n^3 is very pessimistic. Thus multiplication by orthogonal matrices is backward stable. $\square$

Exercise 3.2.72 How can the results of Exercises 3.2.70 and 3.2.71 be improved in the case when Q (or V) is a plane rotator? $\square$

Exercise 3.2.73 The product Qx , where $Q = I - \gamma vv^T$ (with $\gamma = 2/\|v\|_2^2$) is a reflector. In practice there will be slight errors in the computed γ and v , but we will ignore those errors for simplicity. In principle $Qx = y$, where $y = x - \gamma(v^T x)v$.

- (a) Let $\rho = \gamma v^T x$. Using (2.5.3) and Proposition 2.7.1, show that $\text{fl}(\rho) = \rho + \beta$, where $|\beta| \leq 2(n+1)u\|x\|_2/\|v\|_2 + O(u^2)$.
- (b) $\text{fl}(Qx) = \text{fl}(y) = \text{fl}(x - \text{fl}(\rho)v)$. Show that the i th component satisfies $\text{fl}(y_i) = y_i + d_i$, where $|d_i| \leq (2n+7)u\|x\|_2 + O(u^2)$.
- (c) Show that $\text{fl}(Qx) = Q(x + \delta x)$, where $\|\delta x\|_2/\|x\|_2 \leq \sqrt{n}(2n+7)u + O(u^2)$.
- (d) Suppose $A \in \mathbb{R}^{n \times n}$, and we calculate QA by applying the reflector Q to one column of A at a time. Show that $\text{fl}(QA) = Q(A + E)$, where $\|E\|_F/\|A\|_F \leq \sqrt{n}(2n+7)u + O(u^2)$.

□

3.3 SOLUTION OF THE LEAST SQUARES PROBLEM

Consider an overdetermined system

$$Ax = b, \quad A \in \mathbb{R}^{n \times m}, \quad b \in \mathbb{R}^n, \quad n > m. \quad (3.3.1)$$

Our task is to find $x \in \mathbb{R}^m$ such that $\|r\|_2 = \|b - Ax\|_2$ is minimized. So far we do not know whether or not this problem has a solution. If it does have one, we do not know whether the solution is unique. These fundamental questions will be answered in this section. We will settle the existence question affirmatively by constructing a solution. The solution is unique if and only if A has full rank.

Let $Q \in \mathbb{R}^{n \times n}$ be any orthogonal matrix, and consider the transformed system

$$Q^T Ax = Q^T b. \quad (3.3.2)$$

Let s be the residual for the transformed system. Then $s = Q^T b - Q^T Ax = Q^T(b - Ax) = Q^T r$. Since Q^T is orthogonal, $\|s\|_2 = \|r\|_2$. Thus, for a given x , the residual of the transformed system has the same norm as the residual for the original system. Therefore $x \in \mathbb{R}^m$ minimizes $\|r\|_2$ if and only if it minimizes $\|s\|_2$; that is, the two overdetermined systems have the same least squares solution(s).

It seems reasonable to try to find an orthogonal $Q \in \mathbb{R}^{n \times n}$ for which the system (3.3.2) has a particularly simple form [31]. In the previous section we learned that given a square matrix A there exists an orthogonal Q such that $Q^T A = R$, where R is upper triangular. Thus the linear system $Ax = b$ can be transformed to $Q^T Ax = Q^T b$ ($Rx = c$), which can easily be solved by back substitution. The same approach works for the least squares problem, but first we need a QR decomposition theorem for nonsquare matrices.

Theorem 3.3.3 Let $A \in \mathbb{R}^{n \times m}$, $n > m$. Then there exist $Q \in \mathbb{R}^{n \times n}$ and $R \in \mathbb{R}^{n \times m}$, such that Q is orthogonal and $R = \begin{bmatrix} \hat{R} \\ 0 \end{bmatrix}$, where $\hat{R} \in \mathbb{R}^{m \times m}$ is upper triangular, and $A = QR$.

Proof. Let $\tilde{A} = [A \ \tilde{A}] \in \mathbb{R}^{n \times n}$, where $\tilde{A} \in \mathbb{R}^{n \times (n-m)}$ is chosen arbitrarily. Then by Theorem 3.2.20 there exist $Q, \tilde{R} \in \mathbb{R}^{n \times n}$ such that Q is orthogonal, $\tilde{R}$ is upper triangular, and $\tilde{A} = Q\tilde{R}$. We now partition $\tilde{R}$: $\tilde{R} = [R \ \tilde{T}]$, where $R \in \mathbb{R}^{n \times m}$. Then $A = QR$. Since $\tilde{R}$ is upper triangular, R has the form $\begin{bmatrix} \hat{R} \\ 0 \end{bmatrix}$, where $\hat{R} \in \mathbb{R}^{m \times m}$ is upper triangular. $\square$

Since this QR decomposition is obtained by retaining a portion of an n -by- n QR decomposition, any algorithm that computes a square QR decomposition can easily be modified to compute an n -by- m QR decomposition. In practice there is no need to choose a matrix $\tilde{A}$ to augment A , since that portion of the matrix is thrown away in the end anyway. One simply operates on A alone and quits upon running out of columns, that is, after m steps. For example, Algorithm (3.2.45) can be adapted to n -by- m matrices by changing the upper limit on the loop indices to m and deleting the last instruction, which is irrelevant in this case. Q is produced implicitly as a product of m reflectors: $Q = Q_1 Q_2 \cdots Q_m$.

Exercise 3.3.4 Show that the flop count for a QR decomposition of an n -by- m matrix using reflectors is approximately $2nm^2 - 2m^3/3$. (Thus if $n \gg m$, the flop count is about $2nm^2$.) $\square$

The QR decomposition of Theorem 3.3.3 may or may not be useful for solving the least squares problem, depending upon whether or not A has full rank. Recall that the *rank* of a matrix is the number of linearly independent columns, which is the same as the number of linearly independent rows. The matrix $A \in \mathbb{R}^{n \times m}$ ($n \geq m$) has *full rank* if its rank is m ; that is, if its columns are linearly independent. In Theorem 3.3.3 the equation $A = QR$ implies that $\text{rank}(A) \leq \text{rank}(R)$.⁵ On the other hand the equation $R = Q^T A$ implies that $\text{rank}(R) \leq \text{rank}(A)$. Thus $\text{rank}(R) = \text{rank}(A)$, and A has full rank if and only if R does. Clearly $\text{rank}(R) = \text{rank}(\hat{R})$, and $\hat{R}$ has full rank if and only if it is nonsingular. Thus $\hat{R}$ is nonsingular if and only if A has full rank.

Full-Rank Case

Now consider an overdetermined system $Ax = b$ for which A has full rank. Using the QR decomposition, we can transform the system to the form $Q^T Ax = Q^T b$, or

⁵We recall from elementary linear algebra that $\text{rank}(ST) \leq \text{rank}(T)$ for any matrices S and T for which the product ST is defined. As an exercise, you might like to prove this result yourself.

$Rx = c$, where $c = Q^T b$. Writing $c = \begin{bmatrix} \hat{c} \\ d \end{bmatrix}$, where $\hat{c} \in \mathbb{R}^m$, we can express the residual $s = c - Rx$ as

$$s = \begin{bmatrix} \hat{c} \\ d \end{bmatrix} - \begin{bmatrix} \hat{R} \\ 0 \end{bmatrix} x = \begin{bmatrix} \hat{c} - \hat{R}x \\ d \end{bmatrix}.$$

Thus

$$\|s\|_2^2 = \sum_{i=1}^n |s_i|^2 = \|\hat{c} - \hat{R}x\|_2^2 + \|d\|_2^2. \quad (3.3.5)$$

Since the term $\|d\|_2^2$ is independent of x , $\|s\|_2$ is minimized exactly when $\|\hat{c} - \hat{R}x\|_2^2$ is minimized. Obviously $\|\hat{c} - \hat{R}x\|_2^2 \geq 0$ with equality if and only if $\hat{R}x = \hat{c}$. Since A has full rank, $\hat{R}$ is nonsingular. Thus the system $\hat{R}x = \hat{c}$ has a unique solution, which is then the unique minimizer of $\|s\|$. We summarize these findings as a theorem.

Theorem 3.3.6 *Let $A \in \mathbb{R}^{n \times m}$ and $b \in \mathbb{R}^n$, $n > m$, and suppose A has full rank. Then the least squares problem for the overdetermined system $Ax = b$ has a unique solution, which can be found by solving the nonsingular system $\hat{R}x = \hat{c}$, where $\begin{bmatrix} \hat{c} \\ d \end{bmatrix} = c = Q^T b$, $\hat{R} \in \mathbb{R}^{m \times m}$ and $Q \in \mathbb{R}^{n \times n}$ are as in Theorem 3.3.3.*

Exercise 3.3.7 Work this problem by hand. Consider the overdetermined system

$$\begin{bmatrix} 1 \\ 1 \end{bmatrix} \begin{bmatrix} x \end{bmatrix} = \begin{bmatrix} 9 \\ 5 \end{bmatrix},$$

whose coefficient matrix obviously has full rank.

- Before you do anything else, guess the least squares solution of the system.
- Calculate a QR decomposition of the coefficient matrix, where Q is a 2×2 rotator, and R is 2×1 . Use the QR decomposition to calculate the least squares solution. Also deduce the norm of the residual (without calculating the residual directly).

□

When you solve a least squares problem on a computer, the QR decomposition will normally be calculated using reflectors, in which case Q is stored as a product of reflectors $Q_1 Q_2 \cdots Q_m$. Thus the computation $c = Q^T b = Q_m \cdots Q_1 b$ is accomplished by applying reflectors to b using (3.2.40). Recalling that each reflector does a bit less work than the previous one, we find that the total flop count for this step is about $4n + 4(n-1) + 4(n-2) + \cdots + 4(n-m+1) \approx 4nm - 2m^2$. The equation $\hat{R}x = \hat{c}$ can be solved by back substitution at a cost of m^2 flops. It is also advisable to compute $\|d\|_2$ ($2(n-m)$ flops), as this is the norm of the residual

associated with the least squares solution (see (3.3.5)). Therefore $\|d\|_2$ is a measure of goodness of fit. Comparing these flop counts with the result of Exercise 3.3.4, we see that the most expensive step is the computation of the QR decomposition.

Exercise 3.3.8 It is a well-known fact from algebra that every nonzero polynomial of degree $\leq m-1$ is zero at at most $m-1$ distinct points. Use this fact to prove that the matrix in (3.1.3) has full rank. $\square$

Exercise 3.3.8 demonstrates that the least squares problem by a polynomial of degree $\leq m-1$, with $m < n$, always has a unique solution that can be computed by the QR decomposition method.

Exercise 3.3.9 Write a pair of Fortran subroutines to solve the least squares problem for the overdetermined system $Ax = b$, where $A \in \mathbb{R}^{n \times m}$, $n > m$, and A has full rank.

The first subroutine uses reflectors to carry out a QR decomposition, $A = QR$, where $Q \in \mathbb{R}^{n \times n}$ is a product of reflectors: $Q = Q_1 Q_2 \cdots Q_m$, and $R = \begin{bmatrix} \hat{R} \\ 0 \end{bmatrix} \in \mathbb{R}^{n \times m}$, where $\hat{R}$ is m -by- m and upper triangular. All of these operations should be carried out in a single n -by- m array plus one or two auxiliary vectors. If $\hat{R}$ has any zero pivots, the subroutine should set an error flag to indicate that A does not have full rank.

The second subroutine uses the QR decomposition to find x . First $c = Q^T b = Q_m Q_{m-1} \cdots Q_1 b$ is calculated by applying the reflectors successively. An additional one-dimensional array is needed for b . This array can also be used for c (and intermediate results). The solution x is found by solving $\hat{R}x = \hat{c}$ by back substitution, where $c = \begin{bmatrix} \hat{c} \\ d \end{bmatrix}$. The solution should calculate $\|d\|_2$, which is the Euclidean norm of the residual.

Use your subroutines to solve the following problems.

(a) Find the least squares quadratic polynomial for the data

t_i	-1	-0.75	-0.5	0	0.25	0.5	0.75
y_i	1.00	0.8125	0.75	1.00	1.3125	1.75	2.3125

(The correct solution is $\phi(t) = 1 + t + t^2$, which fits the data exactly.)

(b) Find the least squares linear polynomials for the two sets of data

t_i	1000	1050	1060	1080	1110	1130
$(\alpha) y_i$	6010	6153	6421	6399	6726	6701
$(\beta) y_i$	9422	9300	9220	9150	9042	8800

using the basis $\phi_1(t) = 50$, $\phi_2(t) = t - 1065$. Notice that the QR decomposition only needs to be done once. Plot your solutions and the data points.

(c)

$$\begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 1 & 1 & 1 & 1 \\ 3 & 2 & 1 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 10 \\ 26 \\ 42 \\ 4 \\ 6 \end{bmatrix}$$

Examine r_{11} , r_{22} , r_{33} , and r_{44} . You will find that some of them are nearly zero, which suggests that the matrix might not have full rank. In fact its rank is 2. This problem illustrates the fact that your program is unlikely to detect rank deficiency. While two of the r_{ii} should equal zero in principle, roundoff errors have made them all nonzero in practice. On the brighter side, you will notice that the “solution” that our program returned does fit the equations remarkably well. In fact it is (up to roundoff error) a solution. As we shall see, the least squares problem for a rank-deficient matrix has infinitely many solutions, of which your program calculated one.

□

Exercise 3.3.10 MATLAB’s `qr` command, which we introduced in Exercise 3.2.49, can also be used to compute QR decompositions of non-square matrices.

(a) Try out the following commands, for example.

```
n = 6
m = 3
A = randn(n,m)
[Q,R] = qr(A)
Q'*Q
norm(eye(m)-Q'*Q)
norm(A-Q*R)
```

In the interest of convenience, MATLAB returns Q in assembled form.

(b) Write a short MATLAB script (m-file) that solves least squares problems using the `qr` command. (The submatrix $\hat{R}$ can be accessed by writing `R(1:m, 1:m)` or `R(1:m, :)`, for example.) Use your script to solve the least squares problems in parts (a) and (b) of Exercise 3.3.9.

This is not the simplest way to solve least squares problems using MATLAB. Recall from Exercise 3.1.5 that the command `x = A\b` works as well. This gives you a simple way to check your work.

□

Rank-Deficient Case

While most least squares problems have full rank, it is obviously desirable to be able to handle problems that are not of full rank as well. Most such problems can be

solved by a variant of the QR method called the QR decomposition with column pivoting, which we will discuss here. A more reliable (and more expensive) method based on the singular value decomposition will be discussed in Chapter 4.

If A does not have full rank, the basic QR method breaks down in principle because $\hat{R}$ is singular; at least one of its main-diagonal entries $r_{11}, \dots, r_{mm}$ is zero. The QR decomposition with column pivoting makes column interchanges so that the zero pivots are moved to the lower right hand corner of $\hat{R}$. The resulting decomposition is suitable for solving the rank-deficient least squares problem.

Our initial development will ignore the effects of roundoff errors. At step 1 the 2-norm of each column of A is computed. If the j th column has the largest norm, then columns 1 and j are interchanged. The rest of step 1 is the same as before; a reflector that transforms the first column to the form $[-\tau_1, 0, \dots, 0]^T$ is determined. This reflector is then applied to columns 2 through m . Since $|\tau_1|$ equals the 2-norm of the first column, the effect of the column interchange is to make $|\tau_1|$ as large as possible. In particular, $\tau_1 \neq 0$ unless $A = 0$.

The second step operates on the submatrix obtained by ignoring the first row and column. Otherwise it is identical to the first step, except that when the columns are interchanged, the full columns should be swapped, not just the portions that lie in the submatrix. This implies that the effect of the column interchange is the same as that of making the corresponding interchange before the QR decomposition is begun.

If the matrix has full rank, the algorithm terminates after m steps. The result is a decomposition $\hat{A} = QR$, where $\hat{A}$ is a matrix obtained from A by permuting the columns. $R = \begin{bmatrix} \hat{R} \\ 0 \end{bmatrix}$, where $\hat{R}$ is upper triangular and nonsingular. If A does not have full rank, there will come a step at which we are forced to take $\tau_i = 0$. That will happen when and only when all of the entries of the remaining submatrix are zero. Suppose this occurs after r steps have been completed. Letting $Q_i \in \mathbb{R}^{n \times n}$ denote the reflector used at step i , we have

$$Q_r Q_{r-1} \cdots Q_1 \hat{A} = \begin{bmatrix} R_{11} & R_{12} \\ 0 & 0 \end{bmatrix} = R,$$

where R_{11} is $r \times r$, upper triangular, and nonsingular. Its main diagonal entries are $-\tau_1, -\tau_2, \dots, -\tau_r$, which are all nonzero. Clearly $\text{rank}(R) = r$. Letting $Q = Q_1 Q_2 \cdots Q_r$, we see that $Q^T = Q_r Q_{r-1} \cdots Q_1$, $Q^T \hat{A} = R$, and $A = QR$. Since $\text{rank}(A) = \text{rank}(\hat{A}) = \text{rank}(R)$, we have $\text{rank}(A) = r$. We summarize these findings as a theorem.

Theorem 3.3.11 *Let $A \in \mathbb{R}^{n \times m}$ with $\text{rank}(A) = r > 0$. Then there exist matrices $\hat{A}$, Q , and R , such that $\hat{A}$ is obtained from A by permuting its columns, $Q \in \mathbb{R}^{n \times n}$ is orthogonal, $R = \begin{bmatrix} R_{11} & R_{12} \\ 0 & 0 \end{bmatrix} \in \mathbb{R}^{n \times m}$, $R_{11} \in \mathbb{R}^{r \times r}$ is nonsingular and upper triangular, and*

$$\hat{A} = QR.$$

Let us see how we can use this decomposition to solve the least squares problem. Given $x \in \mathbb{R}^m$, let $\hat{x}$ denote the vector obtained from x by making the same sequence

of interchanges to its entries as were made to the columns of A in transforming A to $\hat{A}$. Then $\hat{A}\hat{x} = Ax$, so the problem of minimizing $\|b - Ax\|_2$ is the same as that of minimizing $\|b - \hat{A}\hat{x}\|_2$. An application of Q^T to the overdetermined system $\hat{A}\hat{x} = b$ transforms it to the form $R\hat{x} = Q^T b = c$, or

$$\begin{bmatrix} R_{11} & R_{12} \\ 0 & 0 \end{bmatrix} \begin{bmatrix} \hat{x}_1 \\ \hat{x}_2 \end{bmatrix} = \begin{bmatrix} \hat{c} \\ d \end{bmatrix},$$

where $\hat{x}_1 \in \mathbb{R}^r$ and $\hat{c} \in \mathbb{R}^r$. The residual for this transformed system is

$$s = \begin{bmatrix} \hat{c} - R_{11}\hat{x}_1 - R_{12}\hat{x}_2 \\ d \end{bmatrix},$$

whose norm is

$$\|s\|_2 = \sqrt{\|\hat{c} - R_{11}\hat{x}_1 - R_{12}\hat{x}_2\|_2^2 + \|d\|_2^2}.$$

Clearly there is nothing we can do with the term $\|d\|_2^2$; We minimize $\|s\|_2$ by minimizing $\|\hat{c} - R_{11}\hat{x}_1 - R_{12}\hat{x}_2\|_2^2$. This term can never be negative, but there are many choices of $\hat{x}$ for which it is zero. Each of these $\hat{x}$ is a solution of the least squares problem for the overdetermined system $\hat{A}\hat{x} = b$.

To see how to compute these $\hat{x}$, recall that R_{11} is nonsingular. Thus for any choice of $\hat{x}_2 \in \mathbb{R}^{m-r}$ there exists a unique $\hat{x}_1 \in \mathbb{R}^r$ such that

$$R_{11}\hat{x}_1 = \hat{c} - R_{12}\hat{x}_2.$$

Since R_{11} is upper triangular, $\hat{x}_1$ can be calculated by back substitution. Then $\hat{c} - R_{11}\hat{x}_1 - R_{12}\hat{x}_2 = 0$, and $\hat{x} = \begin{bmatrix} \hat{x}_1 \\ \hat{x}_2 \end{bmatrix}$ is a solution to the least squares problem for $\hat{A}\hat{x} = b$. We have thus established the following theorem.

Theorem 3.3.12 *Let $A \in \mathbb{R}^{n \times m}$ and $b \in \mathbb{R}^n$, $n > m$. Then the least squares problem for the overdetermined system $Ax = b$ always has a solution. If $\text{rank}(A) < m$, there are infinitely many solutions.*

Not only have we proved this theorem, we have also established an algorithm that can be used to calculate any solution of the least squares problem. Thus we have solved the problem in principle.

In practice we often do not know the rank of A . Then part of the problem is to determine the rank. This task is complicated by roundoff errors. After r steps of the QR decomposition with column pivoting, A will have been transformed to the form

$$Q_r \cdots Q_1 \hat{A} = \begin{bmatrix} R_{11} & R_{12} \\ 0 & R_{22} \end{bmatrix},$$

where $R_{11} \in \mathbb{R}^{r \times r}$ is nonsingular. If $\text{rank}(A) = r$, then in principle $R_{22} = 0$ (telling us that the rank is r), and the algorithm terminates. In practice R_{22} will have

been contaminated by rounding errors and will not be exactly zero. Our criterion for determining the rank must take this into account. Thus, for example, we might decide that R_{22} is “numerically zero” if the norm of its largest column is less than $\epsilon \|A\|$, where ϵ is some small parameter depending on the machine precision and the accuracy of the data. This approach generally works well. Unfortunately it is not 100 percent reliable. There exist matrices of the form

$$R = \begin{bmatrix} -\tau_1 & & & * \\ & -\tau_2 & & \\ & & \ddots & \\ 0 & & & -\tau_m \end{bmatrix}$$

that are “nearly” rank deficient, for which none of the $|\tau_i|$ is extremely small. A class of examples due to Kahan is given in [48, p. 31] and also in Exercise 4.2.21. The near rank deficiency of these matrices would not be detected by our simple criterion. A more reliable approach to the detection of rank deficiency is to use the singular value decomposition (Chapter 4).

A few other implementation details need to be mentioned. At each step we need to know the norms of the columns of the remaining submatrix. If these calculations are not done in an economical manner, they can add substantially to the cost of the algorithm.

Exercise 3.3.13 Show that if the norms of the columns are computed in the straightforward manner at each step, the total cost of the norm computations is about $nm^2 - \frac{1}{3}m^3$ flops. How does this compare with the cost of the rest of the algorithm? $\square$

Fortunately the cost can be decreased substantially for steps 2, 3, $\dots$, m by using information from previous steps rather than recomputing the norms from scratch. For example, let us see how we can get the norm information for the second step by using the information from the first step. Let $\nu_1, \nu_2, \dots, \nu_m$ denote the squares of the norms of the columns of A (calculated at a cost of $2nm$ flops). We work with the squares for convenience. For notational simplicity let us assume that ν_1 is the largest (or that the column interchange for step 1 has already been made). After step 1 we have

$$Q_1 A = \left[\begin{array}{c|ccc} -\tau_1 & \tilde{a}_{12} & \cdots & \tilde{a}_{1m} \\ 0 & \tilde{a}_{22} & \cdots & \tilde{a}_{2m} \\ \vdots & \vdots & & \vdots \\ 0 & \tilde{a}_{n2} & \cdots & \tilde{a}_{nm} \end{array} \right].$$

Since Q_1 is orthogonal, it preserves the lengths of vectors. Therefore the norms of the columns of $Q_1 A$ are the same as those of the columns of A . For the second step we need the squares of the norms of the columns of the submatrix

$$\begin{bmatrix} \tilde{a}_{22} & \cdots & \tilde{a}_{2m} \\ \vdots & & \vdots \\ \tilde{a}_{n2} & \cdots & \tilde{a}_{nm} \end{bmatrix}.$$

These can clearly be obtained by the operations

$$\nu_j \leftarrow \nu_j - \tilde{a}_{1j}^2 \quad j = 2, \dots, m.$$

This costs a mere $2(m-1)$ flops instead of the $2(n-1)(m-1)$ flops that would have been required to calculate the squared norms from scratch.

Since the calculation of 2-norms is required, there is some danger of overflow or underflow. This danger can be virtually eliminated by rescaling the entire problem in advance. For example, one can calculate $\nu = \max |a_{ij}|$ and multiply all of the entries of A and b by $1/\nu$. The cost of this scaling operation is $O(nm)$. If this scaling operation is done, it is not necessary to perform scaling operations in the calculation of the reflectors (3.2.37).

Finally we make some observations concerning column interchanges. We have already noted that each column swap requires the interchange of the entire columns. Of course the norm information has to be interchanged too. It is also necessary to keep a record of the interchanges, since the computed solution $\hat{x}$ is the least squares solution of the permuted system $\hat{A}\hat{x} = b$. To get the solution x of the original problem, we must apply the inverse permutation to the entries of $\hat{x}$. This is accomplished by performing the interchanges in the reverse order.

Exercise 3.3.14 Show that after the QR decomposition with column pivoting, the main-diagonal entries of R_{11} satisfy $|\tau_1| \geq |\tau_2| \geq \dots \geq |\tau_r|$. $\square$

Exercise 3.3.15 To get MATLAB's `qr` command to calculate a QR decomposition with column pivoting, simply invoke the command with three output arguments:

$$[Q, R, P] = \text{qr}(A)$$

This gives the Q and R factors plus a permutation matrix P such that $AP = QR$. The effect of the permutation matrix (applied on the right!) is to scramble the columns. Perform the following sequence of MATLAB commands.

```
A = randn(5, 3)
A = A*diag([1 3 9])
[Q, R, P] = qr(A)
```

What was the point of the `diag` command? What does the permutation matrix P tell you? $\square$

3.4 THE GRAM-SCHMIDT PROCESS

In this section we introduce the idea of an orthonormal set, a new formulation of the QR decomposition, and the Gram-Schmidt process for orthonormalizing a linearly independent set. The main result is that performing a Gram-Schmidt orthonormalization is equivalent to calculating a QR decomposition. It follows that the Gram-Schmidt process can be used to solve least squares problems.

A set of vectors $q_1, q_2, \dots, q_k \in \mathbb{R}^n$ is said to be *orthonormal* if the vectors are pairwise orthogonal, and each vector has Euclidean norm 1; that is,

$$\langle q_i, q_j \rangle = \begin{cases} 0 & \text{if } i \neq j \\ 1 & \text{if } i = j. \end{cases}$$

Example 3.4.1 Let $e_1, e_2, \dots, e_n$ be the columns of the identity matrix:

$$e_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix} \quad e_2 = \begin{bmatrix} 0 \\ 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix} \quad \cdots \quad e_n = \begin{bmatrix} 0 \\ \vdots \\ 0 \\ 0 \\ 1 \end{bmatrix}.$$

It is evident that $e_1, e_2, \dots, e_n$ form an orthonormal set. In fact, they form an *orthonormal basis*, since they are also a basis of $\mathbb{R}^n$. From now on we will call $e_1, e_2, \dots, e_n$ the *standard basis* of $\mathbb{R}^n$, and the notation $e_1, \dots, e_n$ will be reserved for this basis. $\square$

Theorem 3.4.2 Let $Q \in \mathbb{R}^{n \times n}$. Then Q is an orthogonal matrix if and only if its columns (rows) form an orthonormal set.

Proof. Let $q_1, q_2, \dots, q_n$ denote the columns of Q . Then

$$Q^T Q = \begin{bmatrix} q_1^T \\ q_2^T \\ \vdots \\ q_n^T \end{bmatrix} \begin{bmatrix} q_1 & q_2 & \cdots & q_n \end{bmatrix} = \begin{bmatrix} q_1^T q_1 & q_1^T q_2 & \cdots & q_1^T q_n \\ q_2^T q_1 & q_2^T q_2 & \cdots & q_2^T q_n \\ \vdots & \vdots & \ddots & \vdots \\ q_n^T q_1 & q_n^T q_2 & \cdots & q_n^T q_n \end{bmatrix}.$$

Thus the entries of $Q^T Q$ are the inner products $\langle q_j, q_i \rangle$. Clearly $Q^T Q = I$ if and only if $q_1, q_2, \dots, q_n$ form an orthonormal set. The analogous theorem for the rows follows from considering the product $Q Q^T$ or from the simple observation that Q^T is orthogonal if and only if Q is. $\square$

Exercise 3.4.3

- Let $A \in \mathbb{R}^{n \times m}$, and let $e_1, \dots, e_m$ denote the standard basis of $\mathbb{R}^m$. Verify that the i th column of A is $A e_i, i = 1, \dots, m$. Thus $A = [A e_1 \ A e_2 \ \cdots \ A e_m]$. This simple observation will be used repeatedly.
- Use the observation of part (a) together with the inner product preserving property of orthogonal matrices to obtain a second proof that the columns of an orthogonal matrix are orthonormal.

$\square$

Using Theorem 3.4.2 as a guide, we introduce a new class of nonsquare matrices that possess some of the properties of orthogonal matrices. The matrix $Q \in \mathbb{R}^{n \times m}$, $n \geq m$ will be called *isometric* (or an *isometry*) if its columns are orthonormal.

Exercise 3.4.4 Prove that $Q \in \mathbb{R}^{n \times m}$ is isometric if and only if $Q^T Q = I$ (in $\mathbb{R}^{m \times m}$). $\square$

The result of this exercise does *not* imply that Q^T is Q^{-1} . Only square matrices can have inverses. It is also *not* true that $Q Q^T$ is the identity matrix.

Exercise 3.4.5 Let $Q \in \mathbb{R}^{n \times m}$ ($n > m$) be an isometry with columns $q_1, q_2, \dots, q_m$.

- (a) Show that $Q Q^T v = 0$ if v is orthogonal to $q_1, q_2, \dots, q_m$.
- (b) Show that $Q Q^T q_i = q_i$, $i = 1, \dots, m$. Therefore $Q Q^T$ behaves like the identity matrix on a proper subspace of $\mathbb{R}^n$.
- (c) Show that $(Q Q^T)^2 = Q Q^T$. Thus $Q Q^T$ is a projector. In fact it is an orthoprojector. (See Remark 3.2.24.)

$\square$

Exercise 3.4.6 Show that if $Q \in \mathbb{R}^{n \times m}$ is an isometry, then

- (a) $\langle Qx, Qy \rangle = \langle x, y \rangle$ for all $x, y \in \mathbb{R}^m$,
- (b) $\|Qx\|_2 = \|x\|_2$ for all $x \in \mathbb{R}^m$.

(Notice that the norm and inner product on the left-hand side of these equations are from $\mathbb{R}^n$, not $\mathbb{R}^m$.) $\square$

Thus isometries preserve inner products, norms, and angles. The converse of both parts of Exercise 3.4.6 holds as well.⁶

The Condensed QR Decomposition

The QR Decomposition Theorem for nonsquare matrices can be restated more elegantly in terms of an isometry.

Theorem 3.4.7 Let $A \in \mathbb{R}^{n \times m}$, $n \geq m$. Then there exist matrices $\hat{Q}$ and $\hat{R}$ such that $\hat{Q} \in \mathbb{R}^{n \times m}$ is an isometry, $\hat{R} \in \mathbb{R}^{m \times m}$ is upper triangular, and

$$A = \hat{Q} \hat{R}.$$

Proof. If $n = m$, this is just Theorem 3.2.20. If $n > m$, we know from Theorem 3.3.3 that there exist matrices $Q \in \mathbb{R}^{n \times n}$ and $R \in \mathbb{R}^{n \times m}$ such that Q is orthogonal, $R = \begin{bmatrix} \hat{R} \\ 0 \end{bmatrix}$, $\hat{R} \in \mathbb{R}^{m \times m}$ is upper triangular, and $A = QR$. Let $\hat{Q} \in \mathbb{R}^{n \times m}$

⁶The name isometry comes from the norm-preserving property: An isometric operator is one that preserves the metric (i.e. the norm in this case).

be the matrix consisting of the first m column of Q . Clearly $\hat{Q}$ is isometric. Let $\tilde{Q} \in \mathbb{R}^{n \times (n-m)}$ be the last $n - m$ columns of Q . Then

$$A = QR = [\hat{Q} \quad \tilde{Q}] \begin{bmatrix} \hat{R} \\ 0 \end{bmatrix} = \hat{Q}\hat{R} + \tilde{Q}0,$$

That is, $A = \hat{Q}\hat{R}$. Since $\hat{Q}$ and $\hat{R}$ have the desired properties, the proof is complete. $\square$

If A has full rank, Theorem 3.4.7 can be strengthened to include the uniqueness of $\hat{Q}$ and $\hat{R}$.

Theorem 3.4.8 *Let $A \in \mathbb{R}^{n \times m}$, $n \geq m$, and suppose $\text{rank}(A) = m$. Then there exist unique $\hat{Q} \in \mathbb{R}^{n \times m}$ and $\hat{R} \in \mathbb{R}^{m \times m}$, such that $\hat{Q}$ is isometric, $\hat{R}$ is upper triangular with positive entries on the main diagonal, and*

$$A = \hat{Q}\hat{R}.$$

The proof is similar to that of Theorem 3.2.46 and is left as an exercise.

Exercise 3.4.9 Prove Theorem 3.4.8. $\square$

In the full-rank case the decomposition of Theorem 3.4.7 or Theorem 3.4.8 can be used to solve the least squares problem for the overdetermined system $Ax = b$. Let $\hat{c} = \hat{Q}^T b$. You can easily check that this vector is the same as the vector $\hat{c}$ of Theorem 3.3.6, and the least squares problem can be solved by solving $\hat{R}x = \hat{c}$ by back substitution. When we use this type of QR decomposition, we do not get the norm of the residual in the form $\|d\|_2$.

A Bit of Review

Before we discuss the Gram-Schmidt process, we should review some of the elementary facts about subspaces of $\mathbb{R}^n$. Recall that a *subspace* of $\mathbb{R}^n$ is a nonempty subset S of $\mathbb{R}^n$ that is closed under the operations of addition and scalar multiplication. That is, S is a subspace of $\mathbb{R}^n$ if and only if whenever $v, w \in S$ and $c \in \mathbb{R}$, then $v + w \in S$ and $cv \in S$. Given vectors $v_1, \dots, v_m \in \mathbb{R}^n$, a *linear combination* of $v_1, \dots, v_m$ is a vector of the form $c_1v_1 + c_2v_2 + \dots + c_mv_m$, where $c_1, c_2, \dots, c_m \in \mathbb{R}$. The numbers $c_1, \dots, c_m$ are called the *coefficients* of the linear combination. In sigma notation a linear combination looks like $\sum_{k=1}^m c_k v_k$. The *span* of $v_1, \dots, v_m$, denoted $\text{span}\{v_1, \dots, v_m\}$, is the set of all linear combinations of $v_1, \dots, v_m$. It is clear that $\text{span}\{v_1, \dots, v_m\}$ is closed under the operations of addition and scalar multiplication; that is, it is a subspace of $\mathbb{R}^n$.

We have already used terms such as *linear independence* and *basis*, but let us now take a moment to review their precise meaning. The vectors $v_1, \dots, v_m$ are *linearly independent* if the equation $c_1v_1 + c_2v_2 + \dots + c_mv_m = 0$ has no solution other than $c_1 = c_2 = \dots = c_m = 0$. In other words, the only linear combination of $v_1,$

$\dots, v_m$ that equals zero is the one whose coefficients are all zero. If the set $v_1, \dots, v_m$ is not linearly independent, then it is called *linearly dependent*.

Every orthonormal set is linearly independent: Let $q_1, \dots, q_m$ be an orthonormal set, and suppose $c_1 q_1 + c_2 q_2 + \dots + c_m q_m = 0$. Taking inner products of both sides of this equation with q_j , we find that

$$c_1 \langle q_1, q_j \rangle + c_2 \langle q_2, q_j \rangle + \dots + c_m \langle q_m, q_j \rangle = \langle 0, q_j \rangle = 0.$$

Since $\langle q_i, q_j \rangle = 0$ except when $i = j$, and $\langle q_j, q_j \rangle = 1$, this equation collapses to $c_j = 0$. This is true for all j , so $c_1 = c_2 = \dots = c_m = 0$. Thus $q_1, \dots, q_m$ are linearly independent. Of course the converse is false; there are lots of linearly independent sets that are not orthonormal.

Exercise 3.4.10 Give examples of sets of two vectors in $\mathbb{R}^2$ that are

- (a) orthonormal. (Think of something other than e_1, e_2 .)
- (b) linearly independent but not orthogonal (hence not orthonormal).
- (c) linearly dependent.

□

Exercise 3.4.11 Prove that the vectors $v_1, \dots, v_m$ are linearly independent if and only if none of them can be expressed as a linear combination of the others; in symbols, $v_j \notin \text{span}\{v_1, \dots, v_{j-1}, v_{j+1}, \dots, v_m\}$ for $j = 1, \dots, m$. □

Let $\mathcal{S}$ be a subspace of $\mathbb{R}^n$, and let $v_1, \dots, v_m \in \mathcal{S}$. Then clearly

$$\text{span}\{v_1, \dots, v_m\} \subseteq \mathcal{S}.$$

We say the $v_1, \dots, v_m$ *span* $\mathcal{S}$ if $\text{span}\{v_1, \dots, v_m\} = \mathcal{S}$. This means that every member of $\mathcal{S}$ can be expressed as a linear combination of $v_1, \dots, v_m$; we say that $v_1, \dots, v_m$ form a *spanning set* for $\mathcal{S}$. A *basis* of $\mathcal{S}$ is a spanning set that is linearly independent. If $v_1, \dots, v_m$ are linearly independent, then they form a basis of $\text{span}\{v_1, \dots, v_m\}$, since they are a spanning set by definition. Recall from your elementary linear algebra course that every subspace of $\mathbb{R}^n$ has a basis; in fact it has many bases. Any two bases of $\mathcal{S}$ have the same number of elements. This number is called the *dimension* of $\mathcal{S}$. Thus, for example, if $v_1, \dots, v_m$ are linearly independent, then $\text{span}\{v_1, \dots, v_m\}$ has dimension m .

The Classical Gram-Schmidt Process

The Gram-Schmidt process is an algorithm that produces orthonormal bases. Let $\mathcal{S}$ be a subspace of $\mathbb{R}^n$, and let $v_1, v_2, \dots, v_m$ be a basis of $\mathcal{S}$. The Gram-Schmidt process takes $v_1, \dots, v_m$ as input and uses them to produce orthonormal vectors $q_1, \dots, q_m$ that form a basis of $\mathcal{S}$. Thus $\mathcal{S} = \text{span}\{v_1, \dots, v_m\} = \text{span}\{q_1, \dots, q_m\}$.

In fact, more is true: The vectors also satisfy

$$\begin{aligned}
 \text{span}\{q_1\} &= \text{span}\{v_1\} \\
 \text{span}\{q_1, q_2\} &= \text{span}\{v_1, v_2\} \\
 \text{span}\{q_1, q_2, q_3\} &= \text{span}\{v_1, v_2, v_3\} \\
 &\vdots \\
 \text{span}\{q_1, q_2, \dots, q_m\} &= \text{span}\{v_1, v_2, \dots, v_m\}.
 \end{aligned} \tag{3.4.12}$$

These relationships are important.

We are given linearly independent vectors $v_1, \dots, v_m \in \mathbb{R}^n$, and we seek orthonormal $q_1, \dots, q_m$ satisfying (3.4.12). In order to satisfy $\text{span}\{q_1\} = \text{span}\{v_1\}$ we must choose q_1 to be a multiple of v_1 . Since we also require $\|q_1\|_2 = 1$, we define $q_1 = (1/r_{11})v_1$, where $r_{11} = \|v_1\|_2$. We know that $r_{11} \neq 0$ because $v_1, \dots, v_m$ are linearly independent, so $v_1 \neq 0$. The equation $q_1 = (1/r_{11})v_1$ implies that $q_1 \in \text{span}\{v_1\}$; hence $\text{span}\{q_1\} \subseteq \text{span}\{v_1\}$. Conversely, the equation $v_1 = r_{11}q_1$ implies $v_1 \in \text{span}\{q_1\}$, and therefore $\text{span}\{v_1\} \subseteq \text{span}\{q_1\}$. Thus $\text{span}\{v_1\} = \text{span}\{q_1\}$.

The second step of the algorithm is to find q_2 such that q_2 is orthogonal to q_1 , $\|q_2\|_2 = 1$, and $\text{span}\{q_1, q_2\} = \text{span}\{v_1, v_2\}$. We will use Figure 3.5 as a guide. In

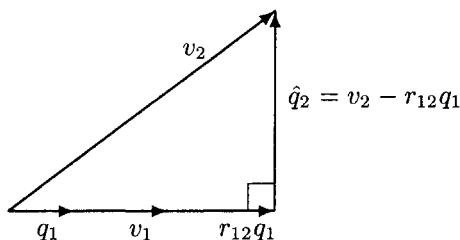


Fig. 3.5 Construction of $\hat{q}_2$.

this figure the space $\text{span}\{v_1, v_2\}$ is represented by the plane of the page. We require q_2 such that q_1, q_2 is an orthonormal set that spans this plane. The figure suggests that we can produce a vector $\hat{q}_2$ that lies in the plane and is orthogonal to q_1 by subtracting just the right multiple of q_1 from v_2 . We can then obtain q_2 by multiplying $\hat{q}_2$ by the appropriate scaling factor. Thus let

$$\hat{q}_2 = v_2 - r_{12}q_1, \tag{3.4.13}$$

where r_{12} is a scalar that should be chosen so that $\langle \hat{q}_2, q_1 \rangle = 0$.

Exercise 3.4.14 Demonstrate that the vector $\hat{q}_2$ given by (3.4.13) satisfies $\langle \hat{q}_2, q_1 \rangle = 0$ if and only if $r_{12} = \langle v_2, q_1 \rangle$. $\square$

Taking $r_{12} = \langle v_2, q_1 \rangle$, we have $\langle \hat{q}_2, q_1 \rangle = 0$. It is clear that $\hat{q}_2 \neq 0$, for if this were not the case, we would have $v_2 = r_{12}q_1 \in \text{span}\{q_1\}$. But the linear independence